

МИНОБРНАУКИ РОССИИ
ФГБОУ ВО «ИжГТУ имени М.Т. Калашникова»
Факультет «Информационные технологии»
Кафедра «Программное обеспечение»

Направление 09.03.04 «Программная инженерия»

УДК 004.6(04)

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к выпускной квалификационной работе бакалавра на тему:
Разработка системы мониторинга программных сервисов

Дипломник
студент гр. Б21-191-2

Н.С. Малых

Руководитель
к.т.н, доцент

И.О. Архипов

Консультант
ст. преподаватель

К.С. Чернышев

Нормоконтролер
ст. преподаватель

В.П. Соболева

Зав. кафедрой ПО
д.э.н, доцент

М.В. Леонов

Ижевск 2025

РЕФЕРАТ

Объем пояснительной записки выпускной квалификационной работы на тему «Разработка мониторинга программных сервисов» – 98 страницы, в том числе 38 рисунков, 12 использованных источника.

Ключевые слова: мониторинг, система, веб-приложение, сервис, метрика, база данных, нотификация, инцидент, опрос.

Целью выпускной квалификационной работы является обеспечение автоматизации сбора метрик и мониторинга сервисов, а также оповещения пользователей об инцидентах. Актуальность темы обусловлена необходимостью быстрого реагирования и исправления неполадок в инфраструктуре компании, чтобы не нести финансовые убытки и потерю репутации для конечного пользователя продуктов компании.

Проектными задачами выпускной квалификационной работы являются:

- 1) реализация реляционной БД отдельно для каждого микросервиса и настройка нереляционной БД (Redis) для микросервиса авторизации,
- 2) реализация серверной части для 3 микросервисов и реализация клиентской части,
- 3) реализация заглушек для сервисов, которые будут находиться под мониторингом системы,
- 4) реализация интеграции микросервиса опросника и микросервиса нотификаций через Kafka,
- 5) реализация интеграции с инструментами мониторинга метрик – Prometheus и Grafana.

При разработке использовались следующие технологии: Java 17, Spring Boot, Spring Security, Spring Data Jpa, PostgreSQL, Hibernate, REST API, Docker, Kafka, Grafana, Prometheus, React JS.

В результате выполненной работы была разработана полнофункциональная веб-система, позволяющая отслеживать состояние

программных сервисов, хранить историю о случившихся инцидентах, управлять параметрами опроса сервисов, контролировать метрики каждого сервиса в Grafana.

Новизна проекта заключается в централизации сбора метрик и мониторинга состояния сервисов с возможностью гибкого масштабирования системы при ее интеграции в инфраструктуру компании за счет микросервисной архитектуры и «api-to-api» взаимодействия.

Разработанное решение может быть применено в любой компании, в которой требуется отслеживать состояние их сервисов, особенно, когда любой сбой может привести к финансовым потерям. Отлично подойдет для небольших компаний и стартапов за счет дешевизны и простоты внедрения.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	7
1. ТРЕБОВАНИЯ К СИСТЕМЕ МОНИТОРИНГА ПРОГРАММНЫХ СЕРВИСОВ	9
1.1. Общее описание продукта.....	9
1.1.1. Назначение объекта автоматизации.....	9
1.1.2. Аналитический обзор	10
1.1.3. Классы и характеристики пользователей	13
1.1.4. Ограничения дизайна и реализации.....	15
1.2. Функциональные требования	16
1.2.1. Микросервис авторизации. Аутентификация пользователя	16
1.2.2. Микросервис авторизации. Авторизация пользователя	16
1.2.3. Микросервис авторизации. Регистрация нового пользователя.....	17
1.2.4. Микросервис авторизации. Выход пользователя из системы	17
1.2.6. Микросервис опроса сервисов. Проверка состояния всех сервисов, доступных пользователю.....	18
1.2.7. Микросервис опроса сервисов. Точечная проверка состояния конкретного сервиса по нажатию на его карточку на клиентской части .	19
1.2.8. Микросервис опроса сервисов. Сортировка сервисов при возврате ответа на клиентскую часть.....	19
1.2.9. Микросервис опроса сервисов. Пагинация и гибкая фильтрация	19
1.2.10. Микросервис опроса сервисов. Настройка интеграции с инструментами мониторинга метрик Prometheus и Grafana	20
1.2.11. Микросервис опроса сервисов. Настройка интеграции с Kafka на стороне микросервиса опроса сервисов.....	20
1.2.12. Микросервис опроса сервисов. Реализация функционала обновления данных о сервисе	21

1.2.13. Микросервис опроса сервисов. Реализация функционала получения данных о сервисе	21
1.2.14. Микросервис опроса сервисов. Реализация функционала фиксации инцидентов и хранения истории о них.....	21
1.2.15. Микросервис нотификаций. Настройка интеграции с Kafka на стороне микросервиса нотификаций.....	22
1.2.16. Микросервис нотификаций. Реализация логики для отправки сообщений на email посредством SMTP-протокола и в Telegram-бота....	22
1.2.17. Микросервис нотификаций. Система должна реализовывать логику защиты от спама и отправки повторных сообщений	23
1.3. Требования к данным	23
1.3.1. Словарь данных	23
1.4. Требования к внешним интерфейсам.....	24
1.4.1. Пользовательский интерфейс	24
1.5. Нефункциональные требования	29
1.5.1. Доступность.....	29
1.5.2. Надежность.....	29
1.5.3. Требования к безопасности.....	29
1.5.4. Конфиденциальность.....	29
1.5.5. Масштабируемость и модульность.....	29
1.5.6. Требования к времени хранения данных.....	29
1.5.7. Требования к производительности	29
2. ПРОЕКТИРОВАНИЕ СИСТЕМЫ МОНИТОРИНГА СЕРВИСОВ.....	30
2.1. Модель сценариев использования предметной области	30
2.2. Логическая модель данных	30
2.3. Структурное моделирование системы.....	34
2.4. Модель классов предметной области	37
2.5. Диаграмма развертывания	40

3. РЕАЛИЗАЦИЯ ПРЕЦЕДЕНТОВ СИСТЕМЫ МОНИТОРИНГА.....	42
3.1. Реализация прецедента «Авторизация и аутентификация».....	42
3.2. Реализация прецедента «Интеграция с системой мониторинга метрик (Grafana, Prometheus)»	48
3.3. Реализация прецедента «Опрос всех сервисов с заданной периодичностью на наличие доступности в текущий момент времени» .	51
3.4. Реализация прецедента «Точечный опрос сервиса на наличие доступности в текущий момент времени».....	55
3.5. Реализация прецедента «Создание инцидента в случае неактивности сервиса и завершение инцидента в случае, когда сервис вновь активен»	58
3.6. Реализация прецедента «Получение детальной информации о сервисе».....	63
3.7. Реализация прецедента «Обновление информации о сервисе»	65
3.8. Реализация прецедента «Получение информации об инцидентах с функциональностью пагинации и гибкой фильтрации»	67
3.9. Реализация прецедента «Отправка сообщений на почту и в телеграм-бота с логикой повторных попыток отправки и защитой от спама»	71
ЗАКЛЮЧЕНИЕ.....	74
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	76
ПРИЛОЖЕНИЕ 1 ТЕКСТЫ ПРОГРАММ.....	77
ПРИЛОЖЕНИЕ 2 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ	98

ВВЕДЕНИЕ

Современное развитие информационных технологий и цифровизация бизнес-процессов предъявляют всё более жёсткие требования к качеству, стабильности и доступности программного обеспечения. С увеличением сложности информационных систем, масштабированием архитектуры и ростом числа пользователей особую значимость приобретает задача обеспечения надёжности и бесперебойной работы программных сервисов. Это особенно актуально для компаний, использующих распределённые системы, в которых задействованы десятки микросервисов, каждый из которых играет важную роль в общем технологическом процессе.

В случае возникновения сбоев в работе отдельных компонентов система может потерять устойчивость, что, в свою очередь, ведёт к серьёзным последствиям: временной недоступности сервисов, потере клиентской базы, убыткам и ухудшению репутации компании. Поэтому задача постоянного мониторинга состояния сервисов и быстрого оповещения ответственных специалистов становится критически важной.

На текущий момент существует ряд решений, предоставляющих возможности мониторинга, однако большинство из них либо слишком сложны во внедрении и сопровождении, либо не обладают достаточной гибкостью и адаптивностью под конкретные бизнес-потребности.

Целью выпускной квалификационной работы является разработка системы мониторинга программных сервисов, обеспечивающей:

- 1) автоматизированный сбор и визуализацию метрик,
- 2) оповещение ответственных лиц об инцидентах,
- 3) возможность анализа производительности и состояния микросервисов в реальном времени.

Актуальность темы обусловлена растущей потребностью IT-компаний в надёжных инструментах контроля и анализа своих систем, а также необходимостью сокращения времени реакции на сбои. Новизна работы заключается в создании расширяемой микросервисной платформы мониторинга, построенной на базе современных технологий с открытым исходным кодом (Spring Framework, Prometheus, Grafana, Kafka), что позволяет адаптировать систему под конкретные нужды компании.

Таким образом, разрабатываемая система мониторинга ориентирована на повышение надёжности и доступности приложений, ускорение устранения неисправностей и улучшение процессов принятия решений в области сопровождения и поддержки программных систем.

1. ТРЕБОВАНИЯ К СИСТЕМЕ МОНИТОРИНГА ПРОГРАММНЫХ СЕРВИСОВ

1.1. Общее описание продукта

1.1.1. Назначение объекта автоматизации

Объектом автоматизации в рамках данной работы является процесс мониторинга микросервисной архитектуры, используемой в современном программном обеспечении. Разрабатываемый продукт предназначен для использования в компаниях, занимающихся разработкой и эксплуатацией распределённых информационных систем. В таких компаниях на постоянной основе функционирует множество микросервисов, каждый из которых реализует определённую бизнес-логику и взаимодействует с другими компонентами системы.

На практике одной из ключевых проблем становится отслеживание состояния этих сервисов, своевременное выявление сбоев и анализ производительности. При отсутствии централизованного мониторинга разработчики и администраторы не способны быстро определить источник неполадки, что приводит к увеличению времени простоя, убыткам и ухудшению пользовательского опыта. Особенно критично это для сервисов с высокой доступностью, где даже кратковременный сбой может иметь значительные последствия.

Существующие бизнес-процессы в компании, не обладающей системой мониторинга, часто включают ручной анализ логов, использование разрозненных инструментов и отсутствие автоматических уведомлений о сбоях. Это создаёт перегрузку на сотрудников, замедляет диагностику и устранение неисправностей. Введение специализированного решения по мониторингу сервисов позволит:

- 1) централизовать сбор и анализ метрик,

- 2) повысить прозрачность и управляемость процессов,
- 3) минимизировать риски длительных простоев,
- 4) оптимизировать нагрузку на технический персонал.

Разрабатываемая система должна представлять собой набор микросервисов, интегрированных с популярными open-source решениями Prometheus и Grafana, что обеспечит гибкость, масштабируемость и визуализацию ключевых метрик в удобной форме. Она также будет включать модуль оповещения, использующий Kafka для передачи уведомлений о критических событиях.

1.1.2. Аналитический обзор

На текущем рынке программных решений представлено множество инструментов для мониторинга. Среди них можно выделить:

1) Zabbix и Nagios — популярные средства мониторинга инфраструктуры. Их основное назначение — отслеживание состояния серверов и сетевых устройств. Однако они слабо адаптированы под нужды приложений, построенных на микросервисной архитектуре. Кроме того, их конфигурация требует значительных усилий (рис.1.1).

Интерфейс Zabbix

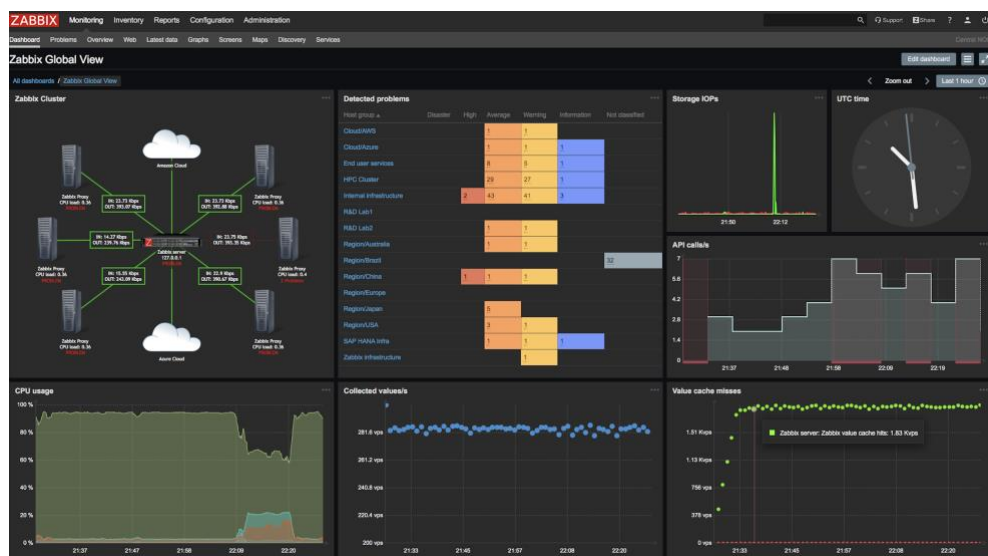


Рис. 1.1

2) Datadog — мощная коммерческая платформа, предоставляющая обширные аналитические функции и широкую интеграцию. Однако высокая стоимость делает её недоступной для многих компаний, особенно малого и среднего бизнеса (рис.1.2).

Интерфейс Datadog

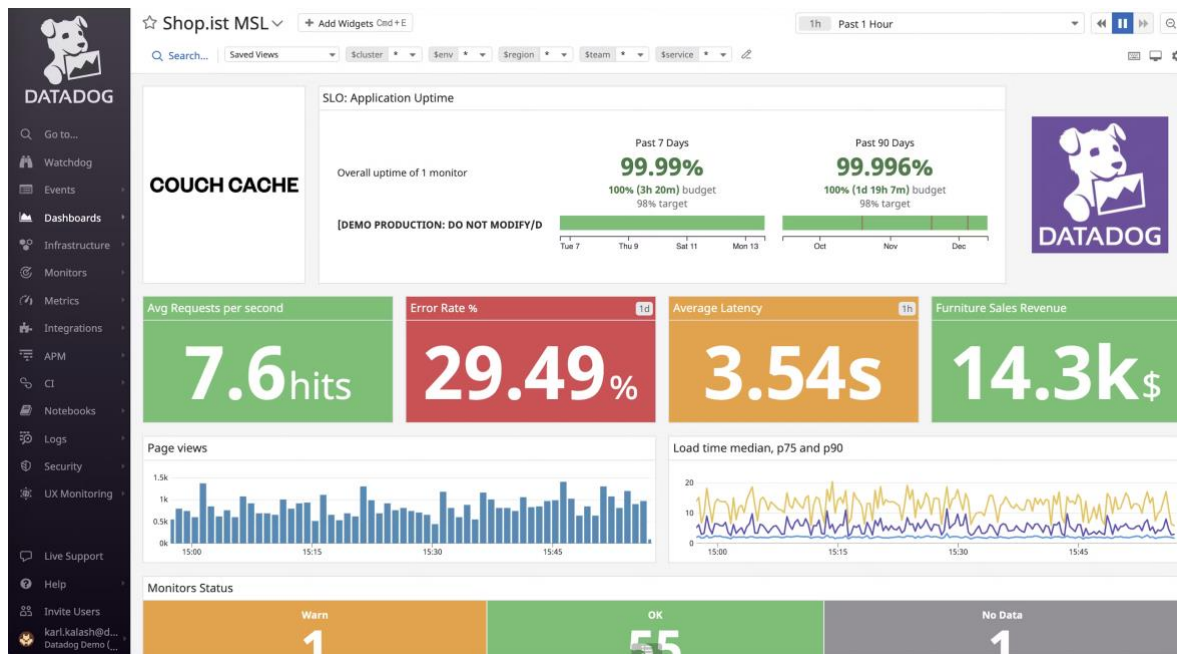


Рис. 1.2

3) Prometheus и Grafana — бесплатные решения с открытым исходным кодом, предназначенные для сбора, хранения и визуализации метрик. Prometheus отличается высокой производительностью и гибкостью, а Grafana предоставляет мощные средства визуализации и настройки дашбордов.

Prometheus и Grafana стали индустриальным стандартом в DevOps-среде благодаря возможности глубокой интеграции, высокой надёжности и расширяемости. Эти инструменты позволяют организовать мониторинг сервисов в реальном времени, отслеживать критические метрики, а также оперативно реагировать на инциденты. Использование связки Prometheus и Grafana в сочетании с собственной микросервисной системой мониторинга предоставляет следующие преимущества:

- 1) модульность архитектуры,
- 2) гибкость масштабирования,
- 3) поддержка кастомных метрик,
- 4) возможность интеграции с системами оповещения,
- 5) быстрая адаптация под конкретные бизнес-потребности.

Вывод: существующие решения, такие как Zabbix и Nagios, не обеспечивают достаточной гибкости и глубины анализа для микросервисных приложений. Коммерческие платформы, в свою очередь, слишком затратны. Это обосновывает целесообразность разработки собственной системы мониторинга, построенной на базе Prometheus и Grafana с учётом требований конкретной компании, а также возможностью дальнейшего расширения.

Таблица 1

Сравнение с конкурентами

Характеристика	Моя система	Zabbix	Datadog
Тип системы	Микросервисная	Монолит	SaaS(облако)
Простота внедрения	Высокая	Средняя	Средняя
Интеграция с Grafana	Полная	Частичная (интеграция вручную)	Отсутствует. Встроенная визуализация
Система уведомлений	Telegram, Email	Email, SMS, Telegram	Slack, Email, Webhooks
Логика хранения истории инцидентов	Имеется	Имеется	Имеется
Расширяемость	Высокая	Ограничена архитектурой	Очень высокая за счет модулей и интеграций

Продолжение табл. 1

Характеристика	Моя система	Zabbix	Datadog
Целевая аудитория	Малые и средние компании, стартапы	Средний и корпоративный сегмент	Средний и корпоративный сегмент
Целевая аудитория	Малые и средние компании, стартапы	Средний и корпоративный сегмент	Средний и корпоративный сегмент

1.1.3. Классы и характеристики пользователей

В рамках проектируемой системы мониторинга программных сервисов предполагается два основных класса пользователей. Каждый из них имеет доступ к определенной функциональности (рис.1.3). Перечислим все классы пользователей:

1) администратор – это пользователь с расширенными правами доступа, выполняющий функции настройки, управления пользователями и поддержки корректной работы системы. Он несет ответственность за администрирование системы, контроль за актуальностью информации о сервисах.

Основные характеристики:

- имеет доступ ко всем функциям системы;
- выполняет настройку и конфигурацию сервисов;
- управляет пользователями: добавление, удаление, редактирование пользователей, изменение ролей;
- отвечает за актуальность и полноту данных о микросервисах;

2) пользователь – это специалист, работающий с системой мониторинга для получения информации о текущем состоянии сервисов и инцидентах. Он может быть как разработчиком, так и менеджером, ответственным за качество и доступность продукта.

Основные характеристики:

- имеет ограниченные права доступа;
- не выполняет изменений в конфигурации системы;
- использует систему для анализа метрик, мониторинга состояния сервисов и оценки их доступности.

Диаграмма вариантов использования

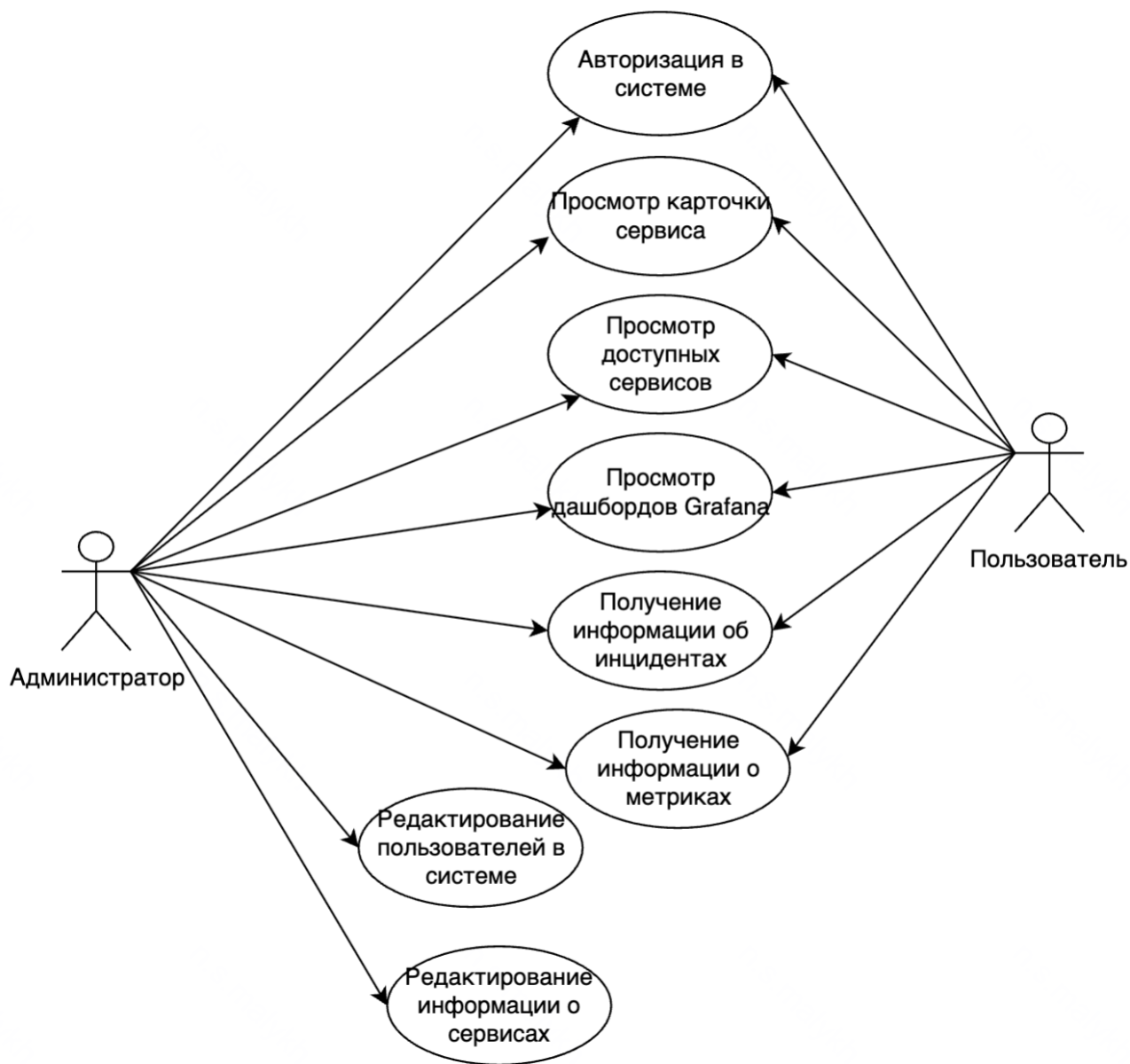


Рис. 1.3

1.1.4. Ограничения дизайна и реализации

Система имеет следующие ограничения дизайна и реализации:

- доступ к клиентской части должен осуществляться только после успешной аутентификации пользователя. При попытке подставить подходящий url в адресную строку будет возврат пользователя на страницу авторизации;
- клиентская часть должна быть реализована на React JS;
- доступ к API на серверной части также должен быть закрыт авторизацией, которая настроена с помощью фреймворка Spring Security;
- серверное приложение должно быть разработано с помощью фреймворка Java Spring Framework и должно иметь микросервисную архитектуру;
- для хранения и записи данных должна быть использована свободная объектно-реляционная система управления базами данных PostgreSQL;
- каждый микросервис должен иметь собственную базу данных PostgreSQL;
- каждый микросервис и клиентская часть, а также все инструменты (Prometheus, Grafana, Kafka) должны быть подняты в Docker-контейнерах и на них должны быть проброшены порты с хоста;
- передача сообщений с микросервиса опросов на микросервис рассылки уведомлений должна происходить через Kafka-server;
- отправка сообщений должна осуществляться через SMTP-протокол на почту, а также в Telegram-бота;
- Telegram-бот должен быть написан на Python;
- Prometheus должен собирать различные метрики (метрики http, баз данных, системные метрики, метрики логов);
- в Grafana должна быть визуализация дашбордов для каждого сервиса.

1.2. Функциональные требования

1.2.1. Микросервис авторизации. Аутентификация пользователя

Приоритет: средний.

Краткое описание: система должна предоставить пользователю возможность пройти аутентификацию с использованием логина и пароля, чтобы получить доступ к ресурсам приложения.

Поведение системы:

- 1) пользователь вводит логин и пароль на клиентской части (интерфейс веб-приложения);
- 2) серверная часть проверяет корректность данных в базе данных (БД);
- 3) если учетная запись существует, то клиентской части возвращается jwt-токен доступа и токен обновления. Токены должны иметь ограниченное время жизни. По истечении срока действия токена обновления пользователь должен пройти повторную аутентификацию;
- 4) Если учетная запись не найдена, то система на клиентской части отображает пользователю сообщение о неверных данных.

1.2.2. Микросервис авторизации. Авторизация пользователя

Приоритет: средний.

Краткое описание: система должна проверять имеет ли аутентифицированный пользователь доступ к выполнению определенных действий в системе.

Поведение системы:

- 1) для каждого запроса, поступающего на серверную часть приложения, проверяется, доступен ли вызов данного api-метода для текущего аутентифицированного пользователя;
- 2) если доступ имеется, то запрос успешно выполняется с кодом ответа 200;
- 3) если доступа нет, то система возвращает пользователю код ответа 403.

1.2.3. Микросервис авторизации. Регистрация нового пользователя

Приоритет: средний.

Краткое описание: система должна предоставлять доступ для регистрации новых пользователей в системе.

Поведение системы:

- 1) пользователь вводит логин и пароль на клиентской части приложения, под которыми он хочет в дальнейшем проходить аутентификацию в системе;
- 2) если такого пользователя еще нет в БД, то информация о новом пользователе сохраняется в базе;
- 3) если такой пользователь уже существует в БД, то система на клиентской части отображает пользователю сообщение о неверных данных.

1.2.4. Микросервис авторизации. Выход пользователя из системы

Приоритет: средний.

Краткое описание: система должна успешно выполнять выход из системы для аутентифицированного пользователя, то есть после выхода из системы, пользователь должен пройти повторную аутентификацию в системе.

Поведение системы:

- 1) аутентифицированный пользователь нажимает на кнопку выхода в клиентской части приложения;
- 2) серверная часть добавляет токен доступа в черный список. Он реализован с помощью кеша в NoSQL Redis;
- 3) каждый запрос, поступающий на серверную часть, проходит фильтр, в котором из заголовка запроса берется токен доступа, и происходит проверка на наличие токена доступа в черном списке;
- 4) если токен доступа находится в черном списке, то пользователю будет возвращен код ответа 403;
- 5) если токен доступа не найден в черном списке, то пользователю будет возвращен код ответа 200.

1.2.5. Микросервис авторизации. Управление пользователями

Приоритет: средний.

Краткое описание: система должна предоставлять администратору возможность управлять пользователями через клиентскую часть приложения.

Поведение системы:

- 1) администратор нажимает на кнопку редактирования пользователей;
- 2) далее он выбирает действие, которое хочет совершить с пользователем в БД;
- 3) вводит предложенные данные в форме и отправляет на серверную часть для дальнейшей обработки и обновления данных о пользователе в БД.

1.2.6. Микросервис опроса сервисов. Проверка состояния всех сервисов, доступных пользователю

Приоритет: высокий.

Краткое описание: система должна регулярно проверять состояние всех сервисов, доступных пользователю, с учётом заданной периодичности, и отображать результаты на клиентской части.

Поведение системы:

- 1) после успешной аутентификации пользователя запускается процесс периодического опроса сервисов, доступных ему;
- 2) основной цикл опроса осуществляется один раз в минуту (настраивается администратором);
- 3) у каждого сервиса в БД задана индивидуальная периодичность опроса;
- 4) при каждом цикле опроса система проверяет, настал ли момент опроса каждого сервиса. Если настал, то выполняется проверка доступности сервиса путем выполнения http-запроса к нему. Если не настал, то проверка этого сервиса пропускается;
- 5) информация о сервисах загружается в кеш при старте приложения, чтобы не обращаться к БД при каждом опросе сервисов.

б) после каждого опроса сервисов информация об их состоянии в кеше обновляется.

1.2.7. Микросервис опроса сервисов. Точечная проверка состояния конкретного сервиса по нажатию на его карточку на клиентской части

Приоритет: высокий.

Краткое описание: система должна уметь обновлять информацию о сервисах точно, не дожидаясь таймера опроса.

Поведение системы:

1) при нажатии на соответствующую кнопку в интерфейсе карточки сервиса на клиентской части приложения должен происходить запрос на проверку его состояния;

2) на серверной части происходит запрос на проверку доступности выбранного сервиса;

3) при этом игнорируется проверка последнего времени опроса.

1.2.8. Микросервис опроса сервисов. Сортировка сервисов при возврате ответа на клиентскую часть

Приоритет: низкий.

Краткое описание: система должна выводить информацию о сервисах на клиентской части в упорядоченном виде, то есть первыми в списке карточек должны быть недоступные сервисы.

Поведение системы:

1) после опроса сервисов список с ними должен быть отсортирован;

2) сортировка должна происходить по полю доступности сервиса. В начало списка должны быть помещены недоступные сервисы.

1.2.9. Микросервис опроса сервисов. Пагинация и гибкая фильтрация

Приоритет: высокий.

Краткое описание: система должна предоставлять возможность выбора фильтров для поиска истории инцидентов (сбоев сервисов) в системе. Данные должны возвращаться постранично.

Поведение системы:

1) пользователь задает фильтры в интерфейсе формы для просмотра инцидентов в клиентской части приложения;

2) на серверной части выполняется sql-запрос к БД на основе этих фильтров и данные в постраничном виде возвращаются на клиентскую часть.

1.2.10. Микросервис опроса сервисов. Настройка интеграции с инструментами мониторинга метрик Prometheus и Grafana

Приоритет: высокий.

Краткое описание: в системе должна быть реализована интеграция с инструментами мониторинга метрик и визуализации метрик – Prometheus и Grafana, соответственно.

Поведение системы:

1) на стороне микросервиса опросника должны быть настроены все конфигурации для Prometheus, чтобы он знал, с каких сервисов ему нужно собирать метрики;

2) также на его стороне должны быть настроены Grafana-дашборды для визуализации метрик;

3) на стороне сервисов, которые находятся под мониторингом, должен быть реализован соответствующий rest-метод, который возвращает метрики.

1.2.11. Микросервис опроса сервисов. Настройка интеграции с Kafka на стороне микросервиса опроса сервисов

Приоритет: высокий.

Краткое описание: система должна отправлять сообщения в брокер сообщений Kafka, чтобы микросервис нотификаций мог их оттуда считывать.

Поведение системы:

1) на стороне микросервиса опросника нужно настроить конфигурацию для отправки сообщений в Kafka-топик;

2) при получении информации о недоступности сервиса сообщение отсылается в Kafka-брокер.

1.2.12. Микросервис опроса сервисов. Реализация функционала обновления данных о сервисе

Приоритет: средний.

Краткое описание: система должна реализовывать функционал, в котором по заданным параметрам в форме клиентской части обновляются соответствующие параметры сервисов.

Поведение системы:

1) пользователь в форме на клиентской части заполняет поля, которые хочет обновить для выбранного сервиса;

2) на серверной части происходит валидация входных данных и в случае успеха выполняется запрос к БД для обновления всех полей.

1.2.13. Микросервис опроса сервисов. Реализация функционала получения данных о сервисе

Приоритет: высокий.

Краткое описание: система должна реализовывать логику сбора данных с таблиц БД о сервисе для ее отображения в карточке сервиса на клиентской части приложения.

Поведение системы:

1) пользователь нажимает на карточку сервиса на клиентской части приложения;

2) на серверной части приложения происходит sql-запрос для сбора информации о сервисе с двух таблиц;

3) информация о сервисе выводится в карточке на клиентской части в виде модального окна.

1.2.14. Микросервис опроса сервисов. Реализация функционала фиксации инцидентов и хранения истории о них

Приоритет: высокий.

Краткое описание: система должна реализовывать функционал фиксации начала и завершения инцидентов, где инцидент – ситуация, когда сервис недоступен.

Поведение системы:

- 1) после получения информации о недоступности сервиса фиксировать начало инцидента в БД со временем его начала и соответствующим статусом;
- 2) если при опросе выяснилось, что сервис вновь активен и в БД есть запись с начатым инцидентом для этого сервиса, то завершить инцидент путем фиксации времени окончания инцидента с соответствующим статусом.

1.2.15. Микросервис нотификаций. Настройка интеграции с Kafka на стороне микросервиса нотификаций

Приоритет: высокий.

Краткое описание: система должна считывать сообщения из брокера сообщений Kafka.

Поведение системы:

- 1) на стороне микросервиса нотификаций нужно настроить конфигурацию для получения сообщений из Kafka-топика;
- 2) при получении сообщения выполнять дальнейшую логику отправки сообщений пользователю.

1.2.16. Микросервис нотификаций. Реализация логики для отправки сообщений на email посредством SMTP-протокола и в Telegram-бота

Приоритет: высокий.

Краткое описание: после получения сообщения из Kafka система должна отправлять его на email и в Telegram-бота параллельно

Поведение системы:

- 1) сообщение успешно получено из топика в Kafka-брокера;
- 2) сообщение прошло валидацию;
- 3) в параллельных потоках запускается отправка сообщений на email по smtp-протоколу и в Telegram-бота.

1.2.17. Микросервис нотификаций. Система должна реализовывать логику защиты от спама и отправки повторных сообщений

Приоритет: высокий.

Краткое описание: система должна обеспечивать защиту от многократных отправок сообщений за короткий промежуток времени с информацией о сбое одного и того же сервиса. Также система должна реализовывать логику попыток повторных отправок сообщений, если по каким-либо причинам не удалось отправить сообщение с первого раза.

Поведение системы:

1) в случае, когда система уже отправила сообщение о недоступности сервиса, нужно фиксировать для него время последней отправки сообщения и игнорировать в ближайшие 30 минут сообщения, полученные из Kafka о недоступности этого сервиса;

2) если при отправке сообщения произошла какая-либо ошибка, то нужно запустить логику повторных попыток отправки сообщения. Она включает в себя 3 повторные попытки отправки сообщения с периодичностью 30 минут. Если одна из этих попыток оказалась успешной, то завершить выполнение логики повторных попыток отправки сообщений.

1.3. Требования к данным

1.3.1. Словарь данных

Сущность Role включает в себя поле name – название роли.

Сущность User включает поля:

- username – логин пользователя,
- password – зашифрованный пароль пользователя,
- role_id – вторичный ключ, связывающий таблицы.

Сущность RefreshToken включает поля:

- token – токен обновления пользователя,

- expiryDate – время истечения токена,
- user_id – вторичный ключ, связывающий таблицы.

Сущность Endpoint включает поля:

- username – логин для межсервисной авторизации,
- password – пароль для межсервисной авторизации,
- role_id – вторичный ключ, связывающий таблицы,
- url – url адрес, по которому доступен сервис,
- period – период опроса сервиса.

Сущность EndpointInfo включает поля:

- name – название сервиса в карточке,
- description – описание сервиса в карточке,
- endpoint_id – вторичный ключ, связывающий таблицы,
- grafana_link – ссылка на Grafana дашборд текущего сервиса.

Сущность Email включает в себя поле reciever – email получателя.

Сущность Endpoint_availability включает поля::

- name – название сервиса, по которому случился инцидент,
- down_time – время начала инцидента,
- up_time – время окончания инцидента,
- summary_down_time – продолжительность инцидента,
- status – текущее состояние сервиса (отключен или активен),
- endpoint_id – вторичный ключ, связывающий таблицы.

Сущность SpanDefender включает поля::

- name – название сервиса, по которому была отправлена нотификация,
- last_notification – время последней отправки нотификации.

1.4. Требования к внешним интерфейсам

1.4.1. Пользовательский интерфейс

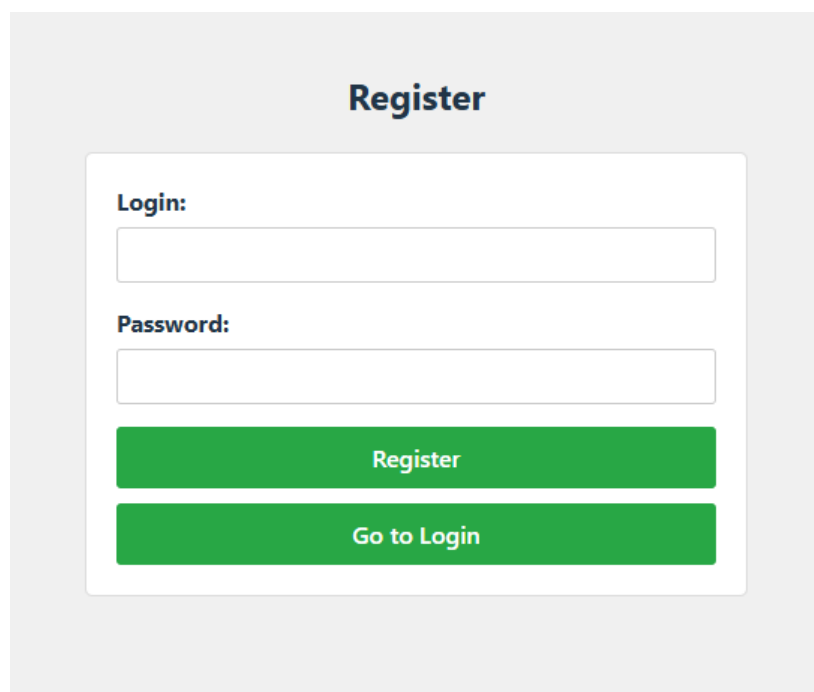
Требования к пользовательскому интерфейсу:

- 1) используются адаптивные компоненты с поддержкой экранов от 1366*768 до 1920*1080 и выше;

- 2) интерфейс реализован с помощью библиотек ReactJS
- 3) цветовая схема выдержана в определенных цветах;
- 4) все кнопки, поля ввода имеют читаемые надписи.

Макеты экранных форм (рис.1.4-1.11):

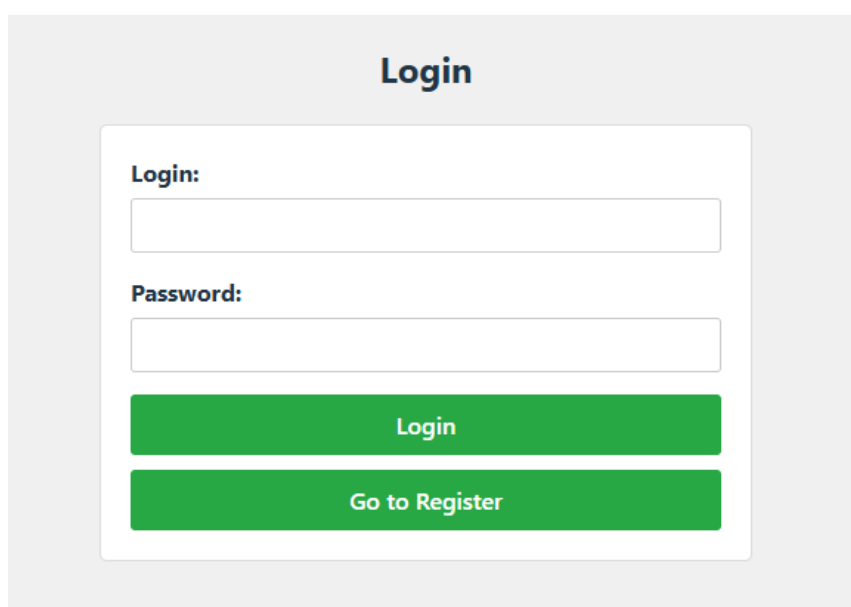
Страница регистрации



The image shows a registration form titled "Register" in a bold, dark blue font. The form is contained within a white box with a thin gray border, set against a light gray background. It features two input fields: "Login:" and "Password:", both with light gray borders. Below the input fields are two green buttons with white text: "Register" and "Go to Login".

Рис. 1.4

Страница логина



The image shows a login form titled "Login" in a bold, dark blue font. The form is contained within a white box with a thin gray border, set against a light gray background. It features two input fields: "Login:" and "Password:", both with light gray borders. Below the input fields are two green buttons with white text: "Login" and "Go to Register".

Рис. 1.5

На основной странице отображается список сервисов в виде карточек с основной информацией о доступности сервисов, связанных с ним. Также есть вкладка с инцидентами и управлением пользователями.

Основная страница со списком сервисов

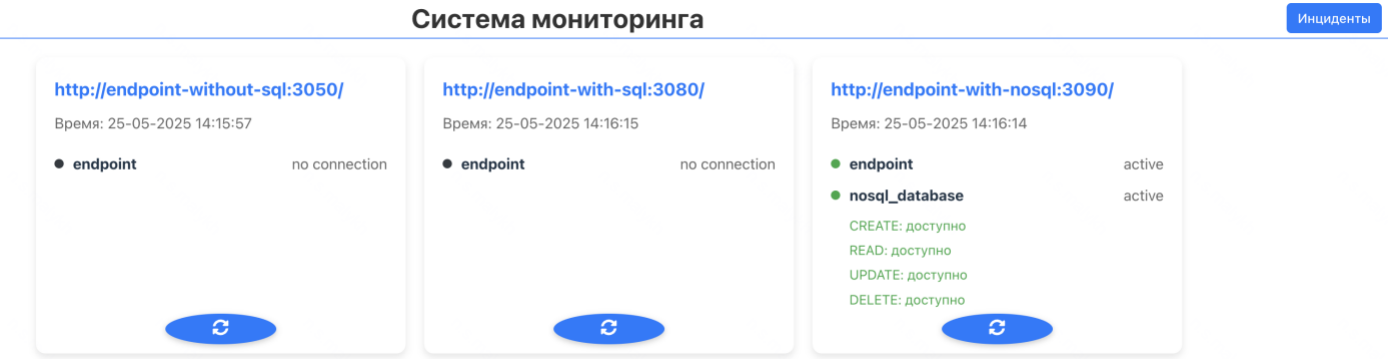


Рис. 1.6

В карточке сервиса отображается описание сервиса. Также в ней есть кнопки для настройки сервиса и перехода в Grafana дашборд для просмотра метрик этого сервиса.

Карточка сервиса

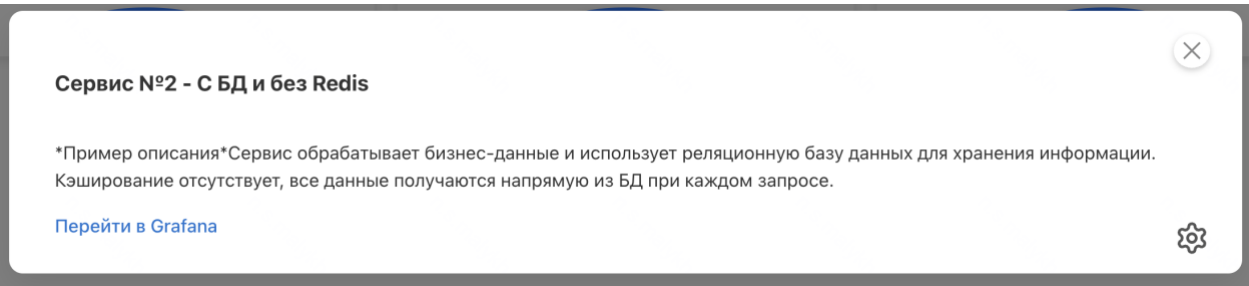


Рис. 1.7

В форме настройки сервиса пользователь может задать новые данные для сервиса. Возможные поля для изменения описаны внутри поля ввода.

Форма настройки сервиса

Сервис №2 - С БД и без Redis

Обновить данные сервиса

URL. Заполните, если необходимо

Период опроса. Заполните, если необходимо

Название сервиса. Заполните, если необходимо

Описание сервиса. Заполните, если необходимо

Ссылка на Grafana. Заполните, если необходимо

Обновить данные сервиса

Назад

Рис. 1.8

В форме истории инцидентов пользователь может просматривать историю инцидентов.

Форма истории инцидентов

Инциденты

Имя

Статус

ДД.ММ.ГГГГ, --:--

ДД.ММ.ГГГГ, --:--

Поиск

Имя	Начало	Конец	Минут
Сервис №2 - С БД и без Redis	2025-05-25T14:15:22.967235		
Сервис №2 - С БД и без Redis	2025-05-25T14:15:22.967235		
Сервис №1 - Без БД	2025-05-25T14:15:08.038334		

Назад

Страница 1 из 1

Вперёд

Рис. 1.9

В Grafana пользователь может просмотреть метрики JVM конкретного сервиса.

Метрики JVM

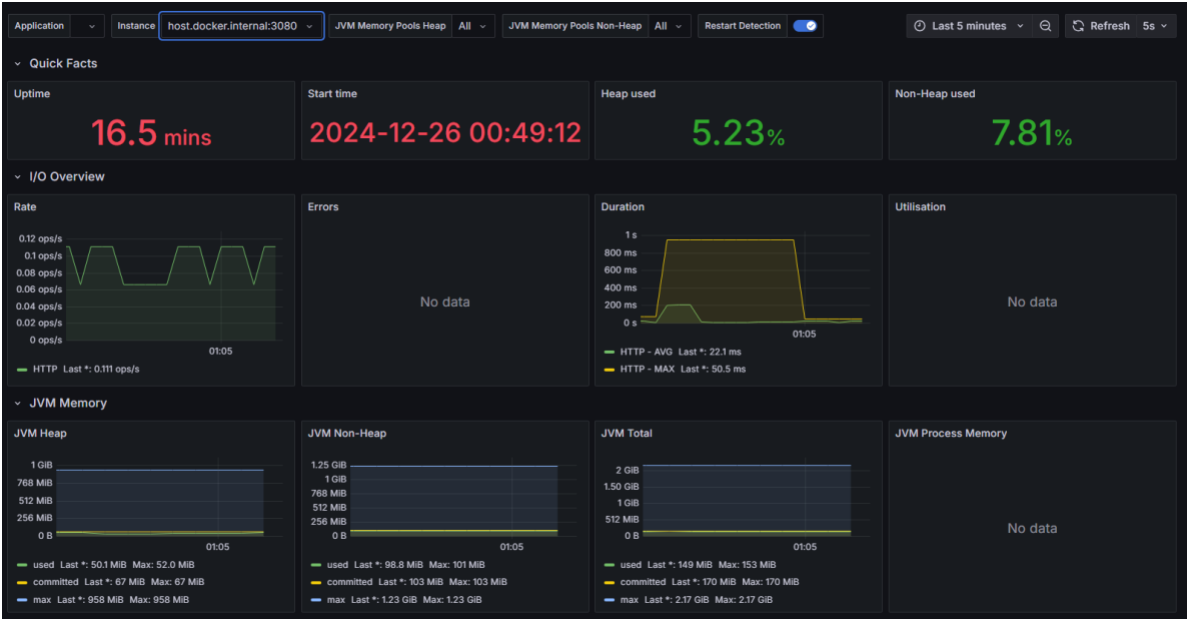


Рис. 1.10

Также в Grafana пользователь может просмотреть метрики фреймворка Spring.

Метрики Micrometr Spring Throughput

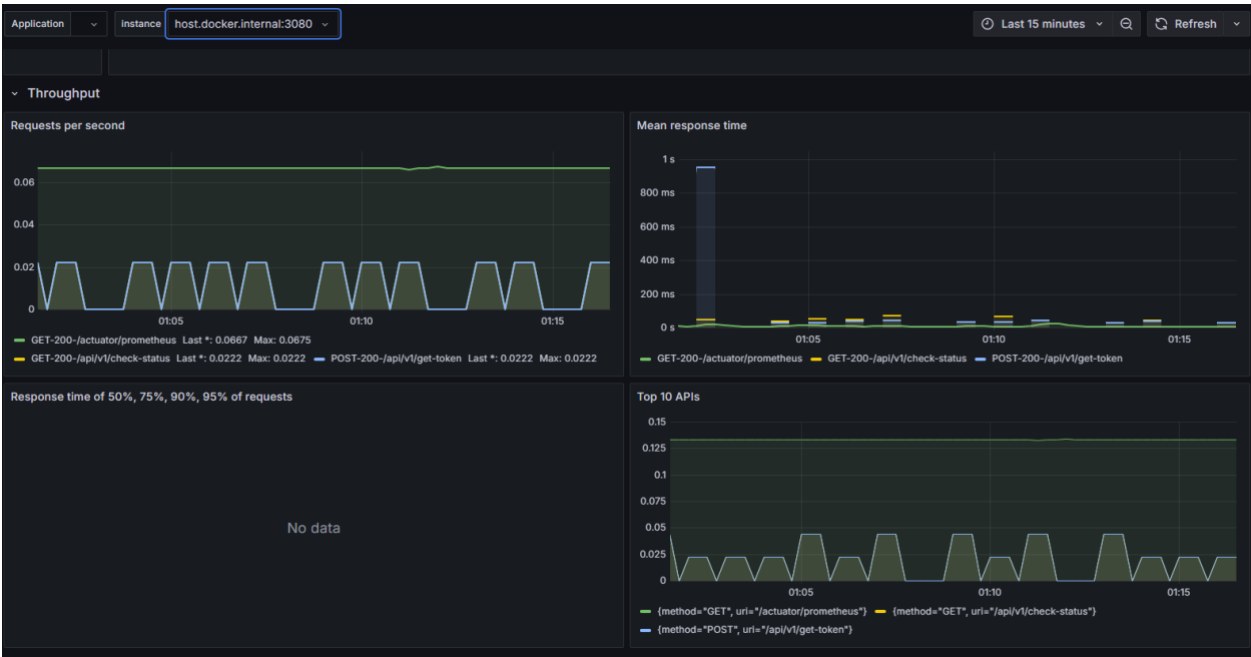


Рис. 1.11

1.5. Нефункциональные требования

1.5.1. Доступность

Веб-сервис должен быть доступен в течение 99% рабочего времени программиста. Клиентская часть приложения должна быть доступна 99% времени дня.

1.5.2. Надежность

В случае возникновения нештатной ситуации, связанной с веб-клиентом может произойти пауза в просмотре состояния сервисов.

1.5.3. Требования к безопасности

Доступ к системе осуществляется только авторизованными пользователями. Управление пользователями в системе осуществляется только пользователем с ролью админ.

1.5.4. Конфиденциальность

Все идентификаторы пользователей в системе должны иметь формат UUID, затрудняющий перебор значений для кражи данных пользователей. Профиль пациента и его личность должны храниться отдельно и быть связаны скрытыми ключами, для того чтобы доступ к ним был только у администратора. Также идентификатор пользователя не должен передаваться по сети.

1.5.5. Масштабируемость и модульность

Серверное приложение должно реализовывать микросервисную архитектуру, для обеспечения модульности и масштабируемости системы.

1.5.6. Требования к времени хранения данных

Данные о сервисах вносятся и удаляются из базы данных вручную. Хранение токенов обновления в базе данные ограничено – задано отдельным полем в таблице.

1.5.7. Требования к производительности

Система должна обеспечивать мониторинг до 1000 сервисов.

2. ПРОЕКТИРОВАНИЕ СИСТЕМЫ МОНИТОРИНГА СЕРВИСОВ

2.1. Модель сценариев использования предметной области

Акторами в системе являются все пользователи, имеющие какую-либо роль в системе и доступ к прецедентам.

Перечисление акторов:

1) пользователь с ролью «user». Роль обычный пользователь – доступ к клиентской части и всему функционалу на нем, просмотру метрик, историческим данным и графикам в Grafana. Он может просматривать только информацию о тех сервисах, к которым ему предоставлен доступ в базе данных;

2) пользователь с ролью «administrator». Роль администратор – доступ к микросервису авторизации и аутентификации с возможностью управления пользователями, полный доступ к клиентской части и просмотру всех дашбордов.

Диаграмма вариантов использования изображена на рис. 1.1.

2.2. Логическая модель данных

Логическая модель данных представляет собой 8 таблиц БД (рис.2.2). Таблица Endpoint является основной таблицей в системе, так как содержит информацию об опрашиваемых сервисах. Таблица EndpointAvailability хранит историю инцидентов. EndpointInfo содержит информацию, которая отображается в карточке. Таблицы Role, User, RefreshToken нужны для реализации авторизации и аутентификации в системе, а также для межсервисной авторизации. Таблицы Email и SpamDefender требуются для реализации логики отправки сообщений по почте и в Telegram-бота в микросервисе нотификаций.

Логическая модель данных

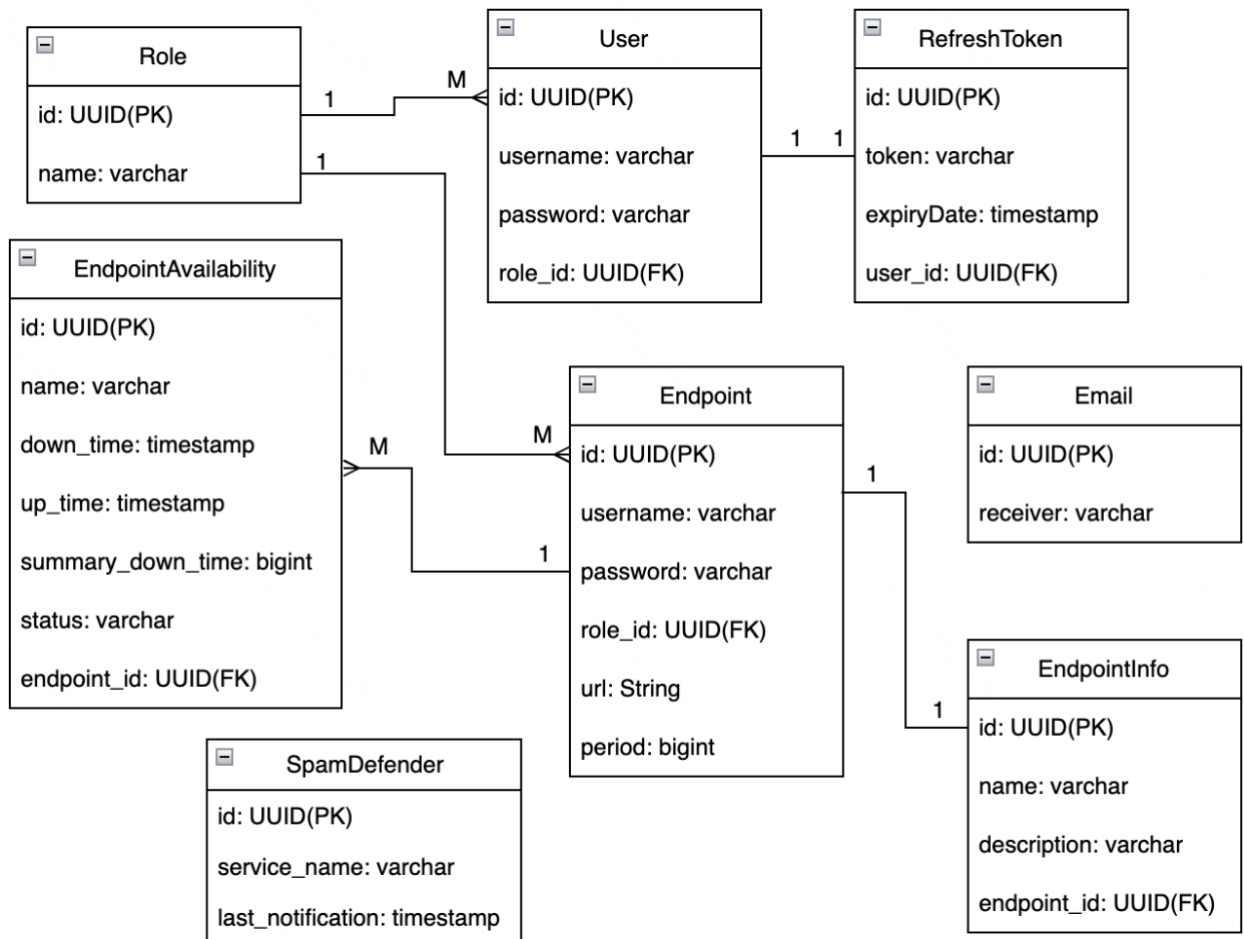


Рис. 2.2

1) Модель данных Пользователь (User) содержит поля:

- первичный ключ Id с типом UUID для обеспечения уникальности каждого элемента таблицы реляционной БД;
- логин пользователя username со строковым типом данных. Нужен для аутентификации пользователя при входе в систему через UI;
- пароль пользователя password со строковым типом данных. Нужен для аутентификации пользователя при входе в систему через UI;
- вторичный ключ role_id с типом данных UUID для идентификации роли пользователя. Отношение Пользователь:Роль имеет связь М:1, то есть многие-к-одному, потому что у одной роли может быть много пользователей, но при этом у одного пользователя по бизнес контексту есть только 1 роль.

2) Модель данных Роль (Role) содержит поля:

- первичный ключ Id с типом UUID для обеспечения уникальности каждого элемента таблицы реляционной БД;

- название роли name со строковым типом данных.

3) Модель данных Токен обновления (RefreshToken) содержит поля:

- первичный ключ id с типом UUID для обеспечения уникальности каждого элемента таблицы реляционной БД;

- токен обновления со строковым типом данных. Нужен для обновления пары токенов (токен доступа и токен обновления);

- время жизни токена с типом данных Instant. Определяет срок жизни токена, то есть до этого времени пользователь может производить аутентификацию без повторного ввода логина и пароля;

- вторичный ключ user_id с типом данных UUID для идентификации пользователя. Отношение Пользователь:Токен_обновления имеет связь 1:1, то есть один-к-одному, потому что за каждым пользователем в любой момент времени закреплен один токен обновления.

4) Модель данных Почта (Email) содержит поля:

- первичный ключ Id с типом UUID для обеспечения уникальности каждого элемента таблицы реляционной БД;

- почта получателя receiver со строковым значением. Необходимо, чтобы определять, на какие адреса отправлять уведомления о сбоях сервисов.

5) Модель данных Сервис (Endpoint) содержит поля:

- первичный ключ Id с типом UUID для обеспечения уникальности каждого элемента таблицы реляционной БД;

- логин сервиса со строковым значением для авторизации к сервису, потому что, как правило, каждый сервис в компании бывает закрыт сервисной авторизацией;

- пароль сервиса со строковым значением для авторизации к сервису, потому что, как правило, каждый сервис в компании закрыт сервисной авторизацией;

- Url-адрес сервиса `url` со строковым типом данных. Необходимо, чтобы определять, по какому адресу доступен сервис;

- период опроса сервиса `period` с типом данных `Instant`. Необходимо, чтобы определять период опроса каждого сервиса. Поле выделено отдельно, потому что у каждого сервиса своя нагрузка и поэтому время опроса целесообразнее сделать разным;

- вторичный ключ `role_id` с типом данных `UUID` для идентификации роли пользователя. Отношение Пользователь:Роль имеет связь М:1, то есть многие-к-одному, потому что у одной роли может быть.

6) Модель данных Карточка сервиса (`EndpointInfo`) содержит поля:

- первичный ключ `Id` с типом `UUID` для обеспечения уникальности каждого элемента таблицы реляционной БД;

- имя сервиса `name` со строковым типом данных. Необходимо, чтобы у сервиса было человекочитаемое название;

- описание сервиса `description` со строковым типом данных. Необходимо, чтобы в карточке сервиса отображалось описание сервиса;

- вторичный ключ `endpoint_id` с типом данных `UUID` для идентификации сервиса. Отношение Сервис:Карточка_сервиса имеет связь 1:1, то есть один-к-одному.

7) Модель данных Доступность сервиса (`EndpointAvailability`) содержит поля:

- первичный ключ `Id` с типом `UUID` для обеспечения уникальности каждого элемента таблицы реляционной БД;

- название сервиса, на котором случился инцидент;

- время начала инцидента (`down_time`). Показывает, в какой момент времени случился инцидент на сервисе.

- время окончания инцидента (`up_time`). Показывает, в какой момент времени закончился инцидент на сервисе, то есть когда сервис вновь стал активен;

- продолжительность инцидента (`summary_down_time`). Показывает продолжительность инцидента сервиса;
- текущее состояние сервиса (`status`). Показывает доступен сервис в текущий момент времени или нет;
- вторичный ключ `endpoint_id` с типом данных `UUID` для идентификации сервиса. Отношение `Доступность_сервиса:Сервис` имеет связь M:1, то есть многие-к-одному, потому что на каждом сервисе может случаться несколько инцидентов.

8) Модель данных Защита от спама (`SpamDefender`) содержит поля:

- первичный ключ `Id` с типом `UUID` для обеспечения уникальности каждого элемента таблицы реляционной БД;
- название сервиса, по которому была отправлена нотификация (`name`);
- последнее время отправки нотификации (`last_notification`).

2.3. Структурное моделирование системы

Система имеет модульную архитектуру (рис.2.3), разделённую на четыре основные подсистемы второго уровня, каждая из которых играет критически важную роль в общей функциональности продукта. Клиентское веб-приложение служит основным интерфейсом для взаимодействия пользователей с системой, обеспечивая интуитивно понятный и отзывчивый интерфейс. Микросервис опросника сервисов выступает в роли центрального механизма для сбора, обработки и анализа данных, предоставляя гибкие инструменты для управления опросами сервисов. Микросервис нотификаций отвечает за своевременное информирование пользователей об инцидентах в сервисах, за которыми ведется мониторинг. Микросервис авторизации обеспечивает безопасный доступ к системе, реализуя механизмы аутентификации и управления правами пользователей.

Структурная схема

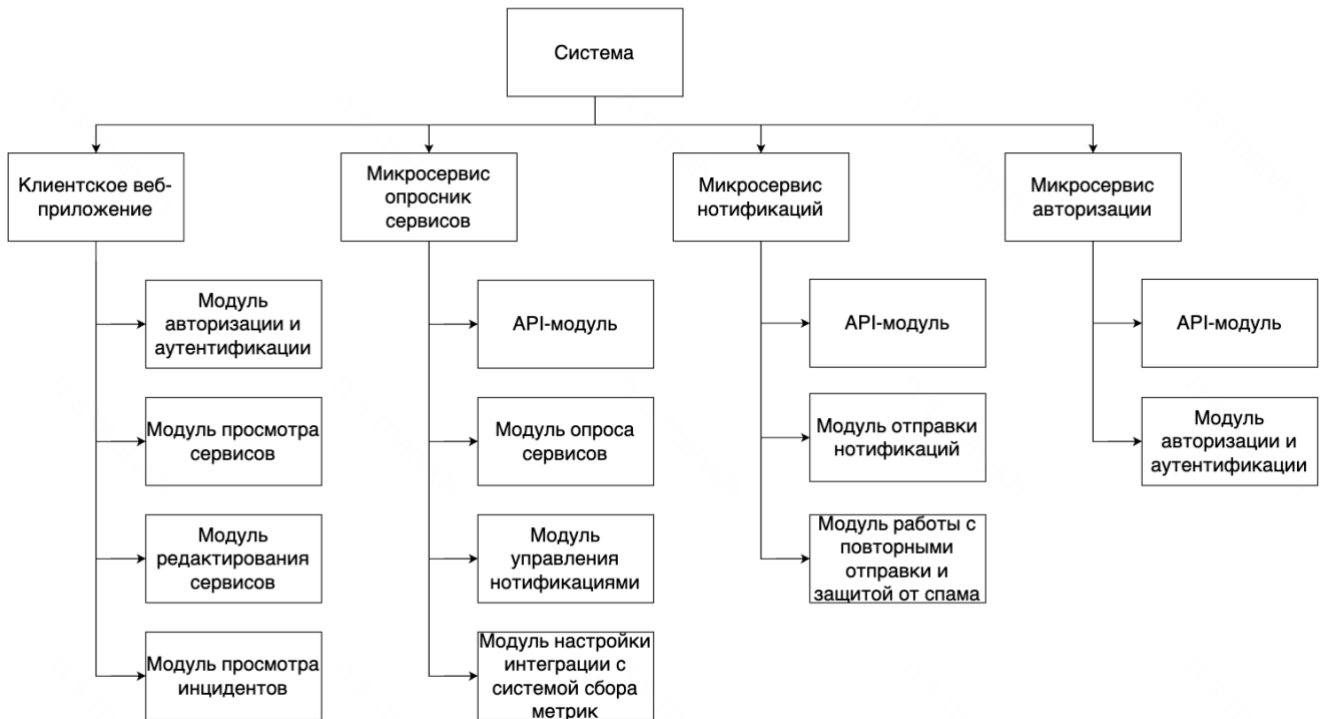


Рис. 2.3

1) Подсистема клиентское веб-приложение предназначена для взаимодействия пользователей с API системы через пользовательский интерфейс. Она предоставляет пользователям функционал, позволяющий просматривать состояние сервисов, управлять пользователями. Содержит три основных модуля:

- модуль авторизации и аутентификации. Позволяет пользователю пройти этап авторизации через UI-интерфейс путем ввода своих данных для авторизации;
- модуль просмотра сервисов. Содержит список опрашиваемых сервисов, которые находятся под мониторингом системы. Позволяет перейти к конкретному сервису, посмотреть его описание и его мониторинг в Grafana;
- модуль работы с пользователями. Представляет собой интерфейс, позволяющий управлять пользователями и сервисами, которые находятся под мониторингом. К этому модулю имеют доступ администраторы.

2) Подсистема микросервис опросник сервисов. Является основным модулем системы. Управляет опросом сервисов, взаимодействует с микросервисом нотификаций, настраивает дашборды в Grafana, собирает метрики. Для реализации функциональности в нем есть четыре модуля:

- API-модуль. Содержит контроллер, который обрабатывает http-запросы, которые поступают в микросервис. Принимает запрос и передает его далее в слой сервиса для дальнейшей обработки бизнес-логики;

- модуль опроса сервисов. С заданной в базе данных периодичностью опрашивает каждый микросервис, и если получены какие либо ошибки от сервиса, то передает эту информацию клиентскому приложению и микросервису нотификаций;

- модуль управления нотификациями. Сообщения в микросервис нотификаций отправляются через брокер сообщений Kafka для обеспечения гарантии доставки;

- модуль настройки сбора метрик и визуализации дашбордов. Отвечает за настройку дашбордов в Grafana и сбора метрик с помощью Prometheus на сервисах, которые опрашиваются системой.

3) Подсистема микросервис нотификаций. Отвечает за получение сообщений из Kafka и отправку их по почте и в телеграм-бота пользователям, которые указаны в базе данных. Состоит из двух модулей:

- API-модуль. Является точкой получения сообщений от микросервиса опросника;

- модуль отправки сообщений. Является точкой обработки сообщений с целью дальнейшей их отправки далее по почте и в телеграм-бота.

4) Подсистема микросервис авторизации. Управляет процессом аутентификации пользователей по ролям, а также выдачей и обновлением токенов безопасности на основе JWT. Содержит два модуля:

- API-модуль. Содержит методы контроллера, которые принимают клиентские данные и передают их далее в слой сервиса для обработки бизнес логики;

- модуль аутентификации и авторизации. Принимает учетные данные пользователя, выполняет проверку данных через базу пользователей и при успешной аутентификации выдает пару токенов – токен обновления и доступа. Также поддерживается механизм блокировки токенов – при выходе пользователя из аккаунта токен добавляется в черный список, поэтому любой запрос сначала происходит проверку наличия токена в черном списке. На основе ролей осуществляется авторизация, то есть предоставление прав доступа пользователю к тому или иному ресурсу.

2.4. Модель классов предметной области

Модель классов реализована в виде 8 основных классов, соответствующих таблицам БД (рис.2.4). Endpoint - центральный класс, хранящий данные об опрашиваемых сервисах. EndpointAvailability фиксирует историю доступности, а EndpointInfo содержит дополнительную информацию для отображения.

Для авторизации используются классы User, Role и RefreshToken, отвечающие за учетные записи, права доступа и механизм обновления токенов. Также сущность Role отвечает за логику межсервисной авторизации.

Нотификации обрабатываются классами Email и SpamDefender. Все классы автоматически мапятся на таблицы БД через ORM, обеспечивая целостность данных и удобную работу в коде. Классы содержат необходимые методы для валидации данных и бизнес-логики соответствующих подсистем. Связи между классами организованы через стандартные механизмы ORM, что упрощает выполнение сложных запросов и поддержку целостности данных при работе распределенной системы.

Модель классов предметной области

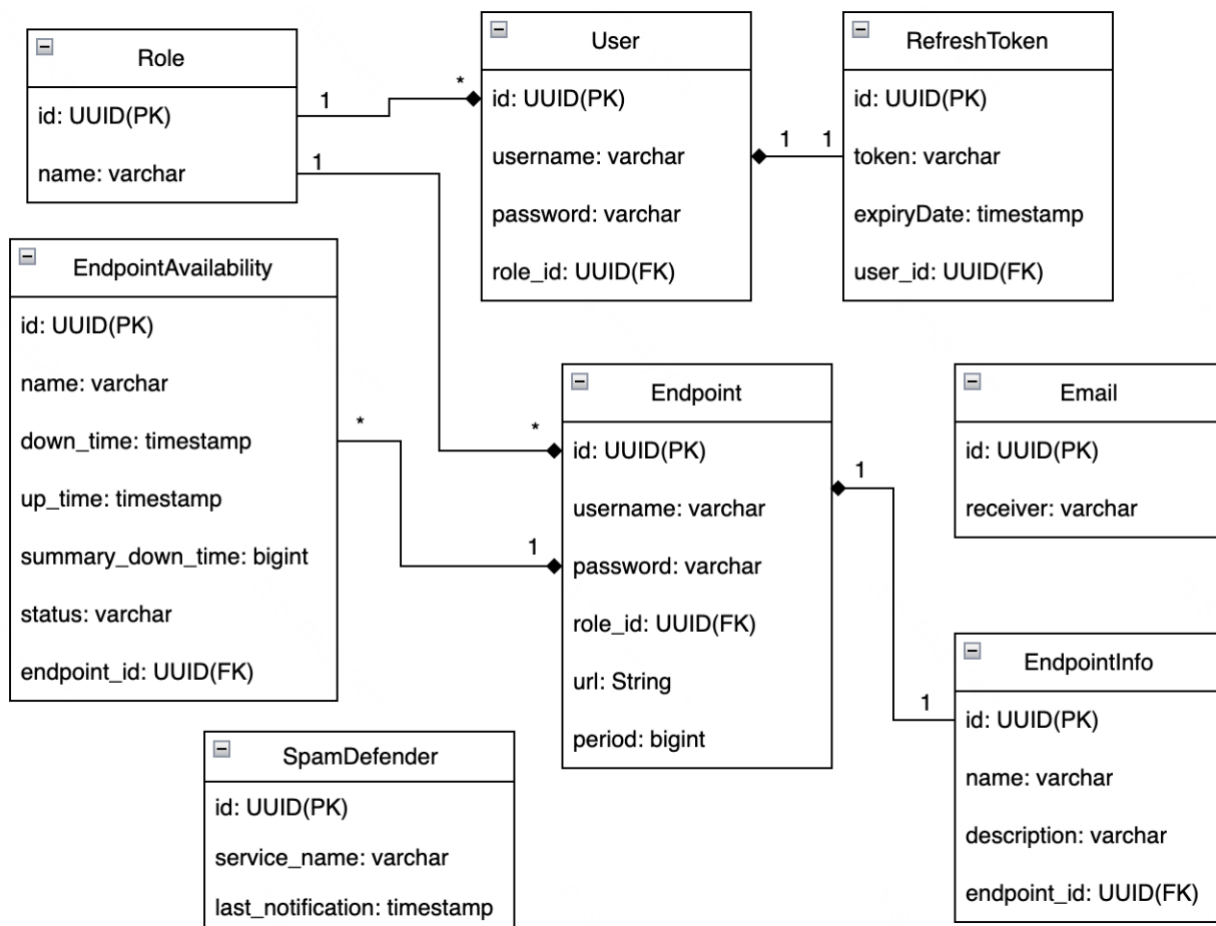


Рис. 2.4

Опишем классы:

1) пользователь (User) с атрибутами:

- id (UUID) — уникальный идентификатор пользователя,
- username (String) — логин для аутентификации,
- password (String) — пароль для аутентификации,
- roleId (UUID) — внешний ключ на роль пользователя.

2) роль (Role) с атрибутами:

- id (UUID) — уникальный идентификатор роли,
- name (String) — название роли («Администратор», «Пользователь»).

3) токен обновления (RefreshToken) с атрибутами:

- id (UUID) — уникальный идентификатор токена,

- token (String) — строка токена обновления,
- expiration (Instant) — время истечения токена,
- userId (UUID) — внешний ключ на пользователя.

4) почта (Email) с атрибутами:

- id (UUID) — уникальный идентификатор записи почты,
- receiver (String) — адрес электронной почты для получения уведомлений.

5) сервис (Endpoint) с атрибутами:

- id (UUID) — уникальный идентификатор сервиса,
- username (String) — логин для авторизации в сервисе,
- password (String) — пароль для авторизации в сервисе,
- url (String) — URL-адрес сервиса,
- period (Instant) — периодичность опроса сервиса,
- roleId (UUID) — внешний ключ на роль.

6) карточка сервиса (EndpointInfo) с атрибутами:

- id (UUID) — уникальный идентификатор карточки,
- name (String) — человекочитаемое имя сервиса,
- description (String) — описание сервиса,
- endpointId (UUID) — внешний ключ на сервис.

7) доступность сервиса (Endpoint_availability) с атрибутами:

- id (UUID) — уникальный идентификатор инцидента,
- name (String) — название сервиса, на котором случился инцидент,
- downTime (LocalDateTime) — время начала инцидента,
- upTime (LocalDateTime) — время окончания инцидента,
- summaryDownTime (Long) — продолжительность инцидента,
- status (String) — текущее состояние активности сервиса,
- endpointId (UUID) — внешний ключ на сервис.

8) защита от спама (SpamDefender) с атрибутами:

- id (UUID) — уникальный идентификатор инцидента,
- name (String) — название сервиса, по которому была отправлена нотификация,
- lastNotification (LocalDateTime) — последнее время отправки нотификации.

2.5. Диаграмма развертывания

На диаграмме развертывания docker-контейнеров схематически изображены взаимосвязи между всеми контейнерами системы (рис.2.5). В верхней части схемы расположена группа контейнеров моей системы мониторинга, включая контейнеры серверной части (бэкенд) и клиентской части (фронтенд), а также контейнеры баз данных каждого микросервиса отдельно и Kafka-сервера. В нижней части расположена группа контейнеров, в которых находятся сервисы, за которыми ведется мониторинг, с ними связаны БД, за которыми также ведется мониторинг. Сначала пользователь входит в систему, происходит проверка его аутентификации. В случае, если пользователь неаутентифицирован, он перенаправляется на страницу входа в систему. Если же он аутентифицирован, то он перенаправляется на главную страницу системы. После входа на главную страницу системы клиентская часть выполняет запрос на серверную часть, чтобы получить информацию о текущем состоянии сервисов. Если сервис недоступен, то отсылается сообщение в kafka-broker, откуда его считывает микросервис нотификаций и отправляет уведомления в почту и telegram-бота. Помимо всего, на микросервисе-опроснике настроен Prometheus и Grafana. Они в связке собирают и визуализируют метрики сервисов, находящихся под мониторингом системы.

Диаграмма разворачивания docker-контейнеров системы мониторинга программных сервисов

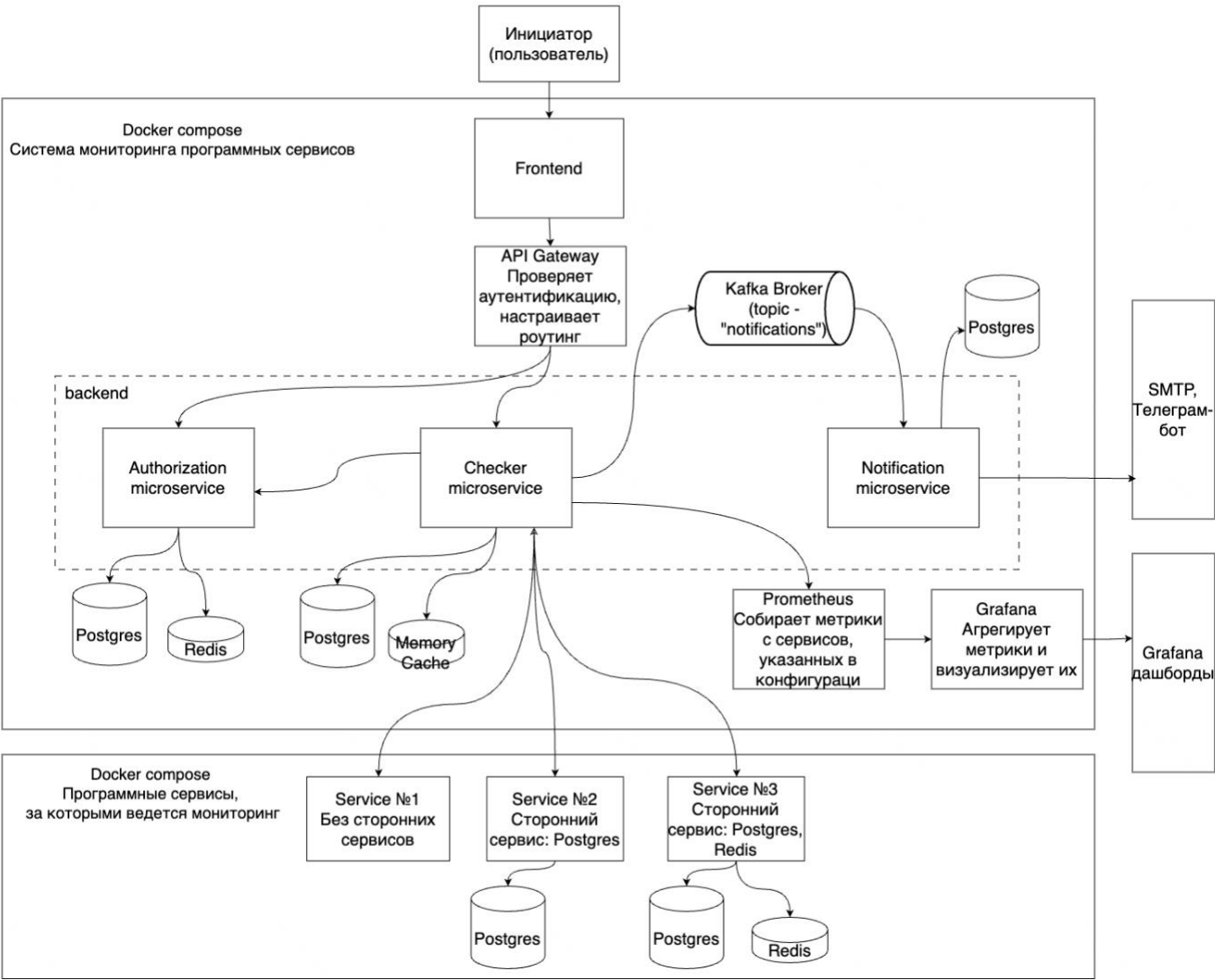


Рис. 2.5

3. РЕАЛИЗАЦИЯ ПРЕЦЕДЕНТОВ СИСТЕМЫ МОНИТОРИНГА

3.1. Реализация прецедента «Авторизация и аутентификация»

Пользователь вводит логин и пароль для регистрации на форме регистрации. Для обработки запроса реализован контроллер в микросервисе авторизации и аутентификации. Происходит валидация входных данных, проверка на наличие пользователя с таким логином в БД. В случае, если валидации не пройдены или пользователь уже существует, на форме регистрации появится соответствующее информативное сообщение. Иначе, пользователь сохраняется в БД (рис.3.1).

Пользователь вводит логин и пароль для входа в систему на соответствующем форме на клиентской части. Для обработки запроса реализован контроллер в микросервисе авторизации и аутентификации. Сначала происходит валидация входных данных. Если она не пройдена, то на форме входа в систему отобразится соответствующее сообщение. Иначе, пользователь будет аутентифицирован, будут сгенерированы токены доступа и обновлена и возвращены на клиентскую часть. Клиентская часть на своей стороне хранит пару токенов и при каждом запросе к сервису авторизации и аутентификации передает в заголовках запроса токен доступа (рис.3.2).

В случае, если токен доступа истек, нужно сгенерировать новый токен, чтобы избежать повторной авторизации, так как токен обновления еще не истек. Для этого реализован контроллер в микросервисе авторизации и аутентификации. Он возвращает пару токенов, где токен доступа обновленный, а токен обновления прежний (рис.3.3).

Если истечет срок жизни токена обновления, то пользователь автоматически попадет на страницу авторизации, то есть его текущая активная сессия прервется и ему придется заново пройти этап авторизации для выдачи новой пары токенов.

Для выхода из системы реализован контроллер в микросервисе авторизации и аутентификации. Он извлекает из заголовков запроса токен доступа и помещает его в черный список. Он, в свою очередь, реализован через кеш (Redis). Также все запросы к сервису авторизации и аутентификации проходят через фильтр, в котором проверяется, находится ли токен в текущем запросе в черном листе. Если да, то запрос отклоняется. Иначе, запрос проходит.

3.1.1. Реализация алгоритма

Схема «Регистрация пользователя»

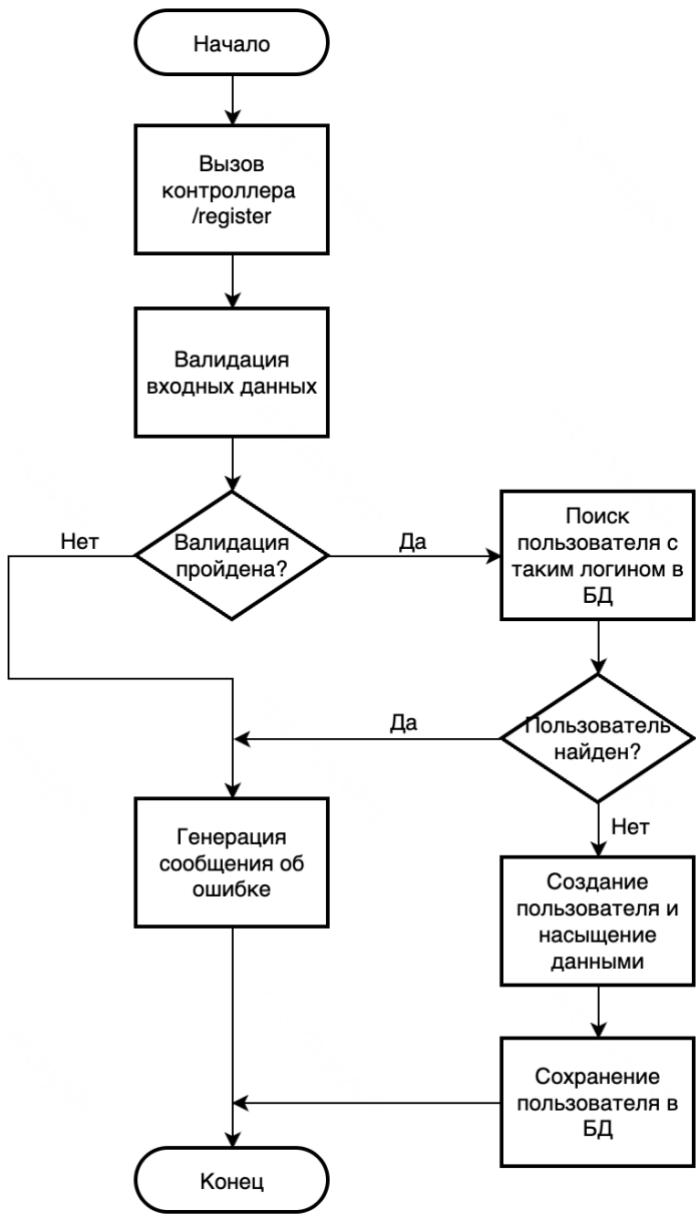


Рис. 3.1

Схема «Авторизация пользователя»

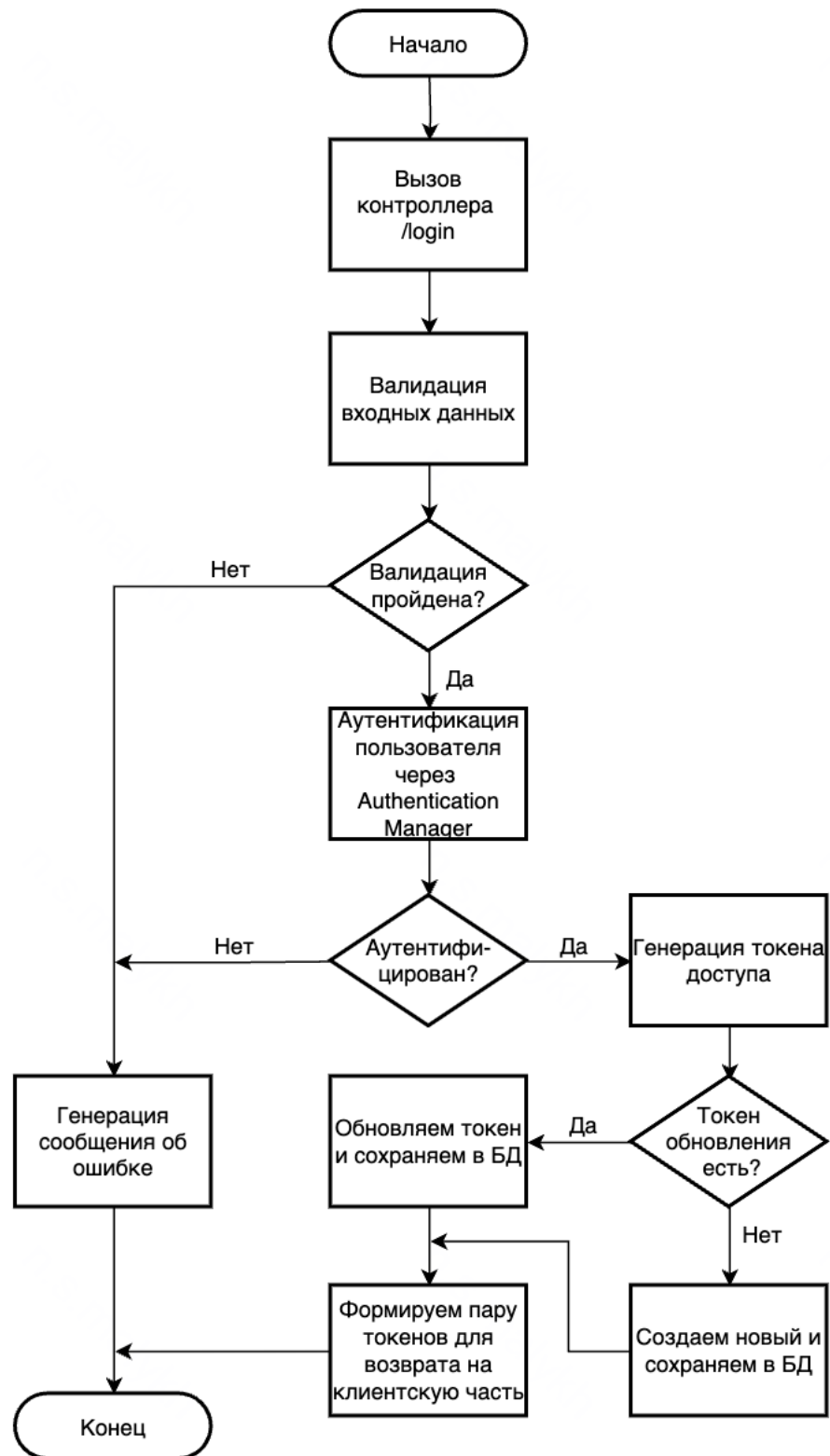


Рис. 3.2

Схема «Обновление токена доступа»

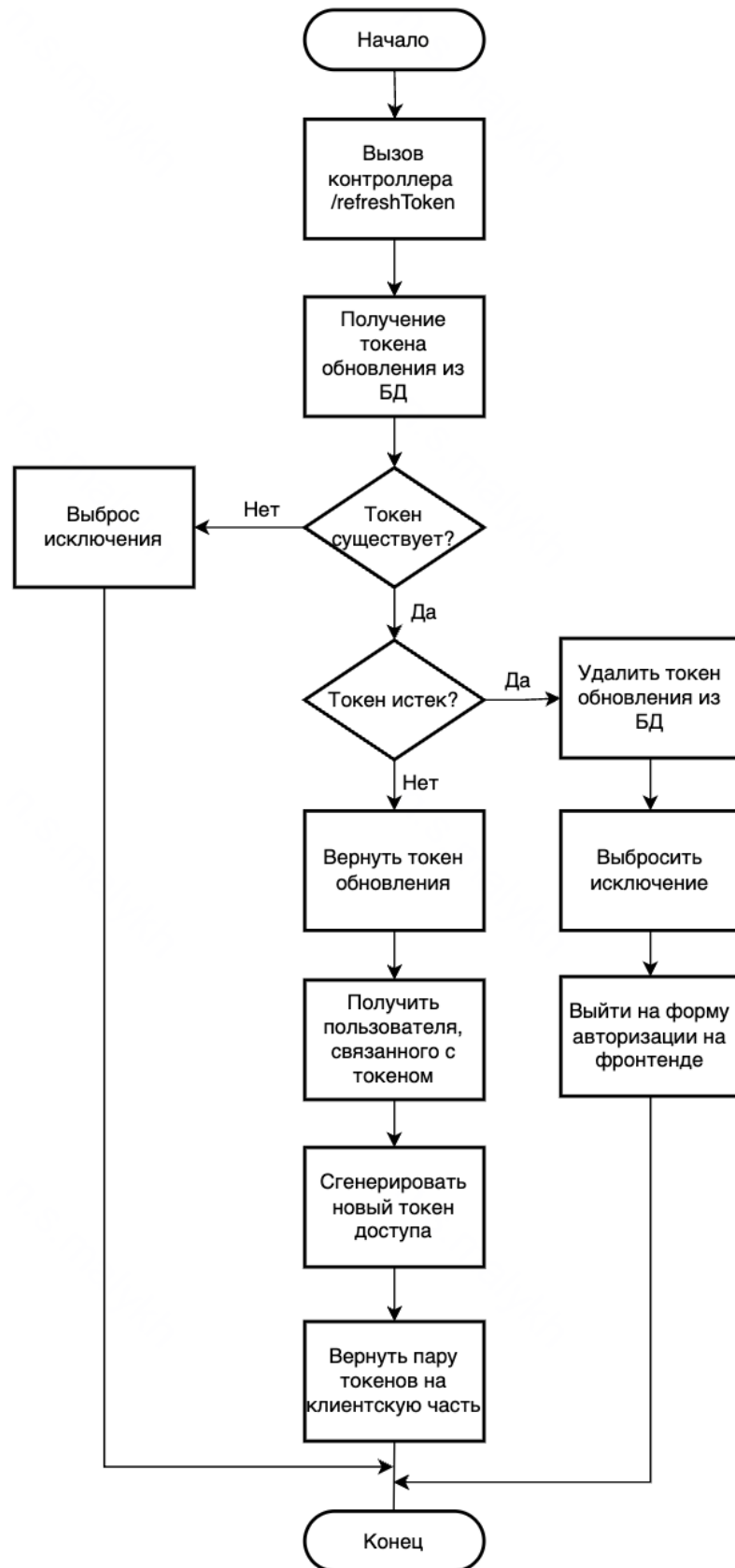


Рис. 3.3

3.1.2. Реализация классов

Класс `AuthController` предназначен для реализации api-взаимодействия, то есть в нем реализованы rest-методы.

Сводка полей класса:

- `JwtService` – сервис для работы с токеном доступа;
- `AuthenticationManager` – библиотечный класс фреймворка `Spring Security`, предназначенный для аутентификации пользователя;
- `UserService` – сервис для управления пользователя в системе;
- `RefreshTokenService` – сервис для работы с токеном обновления;
- `RedisTokenBlackList` – сервис, реализующий методы для кеширования в `Redis` для реализации черного списка токенов доступа.

Сводка rest-методов класса:

- `authenticationAndGetToken(SignInDto signInDto)` – rest-метод для авторизации пользователя;
- `logout(HttpServletRequest request)` – rest-метод для реализации выхода пользователя из системы;
- `registerUser(SignUpDto signUpDto)` – rest-метод для реализации сохранения нового пользователя в БД;
- `refreshToken(RefreshTokenRequestDto refreshTokenRequestDto)` – rest-метод для обновления токена доступа и токена обновления;
- `getCurrentRole()` – rest-метод для получения роли текущего аутентифицированного пользователя в системе.

Класс `JwtService` содержит поле `SECRET` – специальный секретный ключ для шифрования jwt-токена.

Сводка методов класса `JwtService`:

- `extractUsername(String token)` – извлекает из jwt-токена информацию об `username`;
- `extractExpiration(String token)` – извлекает из jwt-токена информацию об `expirationDate`;

- `isTokenExpired(String token)` – проверяет истек ли токен;
- `createToken(Map claims, String username)` – создает токен со всеми «claims» и сроком жизни.

Класс `UserService`.

Сводка полей класса:

- `UserRepo` – репозиторий для взаимодействия с таблицей пользователей в БД;
- `RoleRepo` – репозиторий для взаимодействия с таблицей ролей в БД;
- `PasswordEncoder` – библиотечный класс фреймворка Spring-security.

Класс `UserService` содержит метод `save(SignUpDto signUpDto)` – метод, реализующий сохранение нового пользователя в таблице БД.

Класс `RefreshTokenService`.

Сводка полей класса:

- `RefreshTokenRepository` – репозиторий для взаимодействия с таблицей БД, хранящей токены обновления пользователей;
- `UserRepo` – репозиторий для взаимодействия с таблицей пользователей в БД;

Сводка методов класса:

- `createRefreshToken()` – создать новый токен обновления;
- `verifyExpiration(RefreshToken token)` – проверить истек ли срок жизни токена обновления.

3.1.3. Описание контрольного примера

Выход пользователя из системы. Добавление токена в черный список



Рис. 3.6

Регистрация нового пользователя

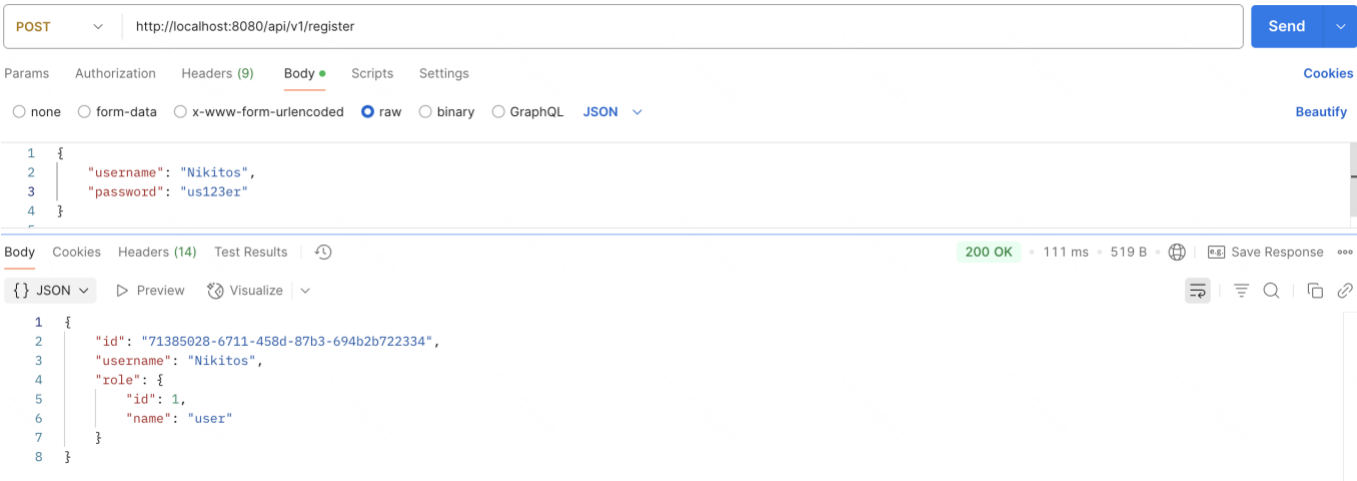


Рис. 3.4

Авторизация нового пользователя

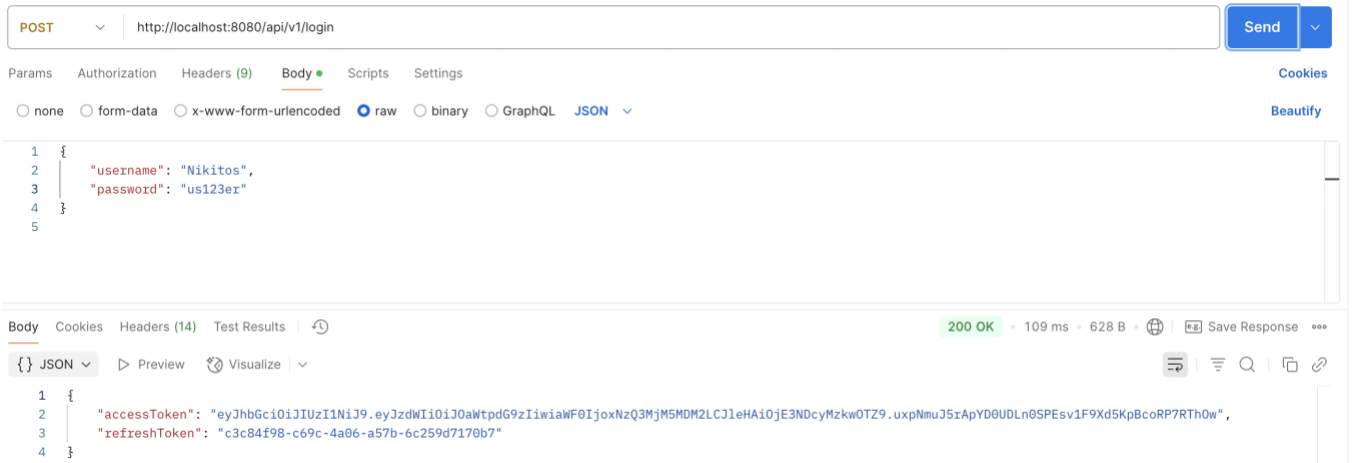


Рис. 3.5

3.2. Реализация прецедента «Интеграция с системой мониторинга метрик (Grafana, Prometheus)»

Обеспечивает регулярный сбор, агрегацию и визуализацию количественных характеристик(метрик) состояния программных сервисов с последующим отображением этих данных в системе визуализации.

3.2.1. Математическая постановка

Изначально дано множество сервисов:

$$S = \{s_1, s_2, s_3, \dots\}, \tag{1}$$

где s_i – сервис, за которым ведется мониторинг. Каждый из них является экземпляром программы или инфраструктурной единицы системы (БД, брокер сообщений и т.п.).

Prometheus осуществляет выборку значений метрик у сервисов s_i через HTTP с интервалом Δt секунд. Собранные данные формируют дискретное отображение – сбор метрик во времени (дискретизация):

$$M_i = \{(t_k, \vec{u}_i(t_k)) | t_k = t_0 + k * \Delta t, k \in \mathbb{N}\}, \quad (2)$$

где t_k – это каждый момент времени, когда Prometheus делает опрос (каждые 15 секунд)

Δt – интервал опроса

$\vec{u}_i(t_k)$ – вектор значений метрик на момент времени t_k

$$E_i: \mathbb{T} \rightarrow \mathbb{R}^m, \quad (3)$$

где E_i – функция, которую представляет API-метод /actuator/prometheus у сервиса s_i

$\mathbb{T}$ – множество моментов времени

$\mathbb{R}^m$ – m измерений: это количество различных метрик (например: CPU, память, HTTP-запросы)

Временные ряды:

$$TS_{i,j} = \{(t_k, u_{i,j}(t_k))\}, \quad (4)$$

где $TS_{i,j}$ – это временной ряд для одной метрики j сервиса s_i (например: $TS_{1,1}$ – это все значения CPU у первого сервиса за все время).

Агрегация метрик (построение графиков в дашбордах в Grafana):

$$Q_{i,j}^\delta = \{(t, A_{[t, t+\delta]}(u_{i,j})) | t \in [t_a, t_b]\}, \quad (5)$$

где $[t_a, t_b)$ – диапазон времени, который отображается на графике (например, последние 6 часов)

δ – длина окна агрегации (например, 60 секунд)

$A_{[t, t+\delta]}(u_{i,j})$ – агрегирующая функция (например, среднее значение за интервал)

В результате формируется набор точек на графике, где каждая точка – результат агрегации значений за окно δ .

3.2.2. Реализация алгоритма

Алгоритм заключается в экспортировании метрик с сервисов, которые находятся под мониторингом, их агрегацией и дальнейшей визуализацией (рис.3.7).

Схема прецедента «Интеграция с системой мониторинга метрик (Grafana, Prometheus)»

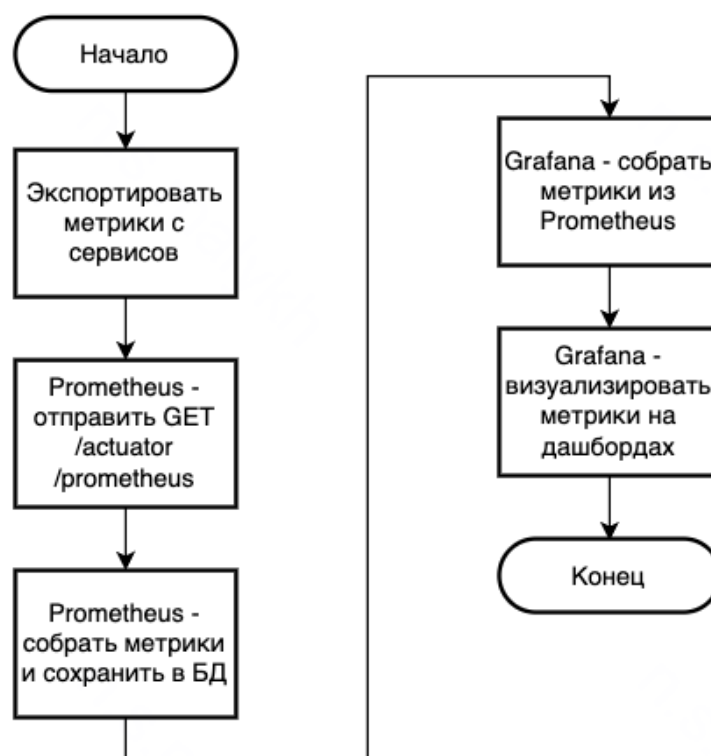


Рис. 3.7

3.2.3. Описание контрольного примера

Исходные данные:

- 1) тестовые сервисы реализуют API-метод /actuator/prometheus, возвращающий стандартные метрики Spring-boot;
- 2) Prometheus настроен с конфигурацией prometheus.yml;
- 3) Grafana подключена к Prometheus и имеет настроенный дашборд с панелью, которая визуализирует метрики из Prometheus.

Порядок действий системы и пользователя:

- 1) запускаются сервисы;
- 2) Prometheus собирает значения метрик;
- 3) Grafana агрегирует данные из Prometheus и визуализирует их на дашбордах;
- 4) пользователь переходит в Grafana через интерфейс системы мониторинга программных сервисов и смотрит результат агрегации и визуализации значений метрик.

Дашборд метрик сервиса

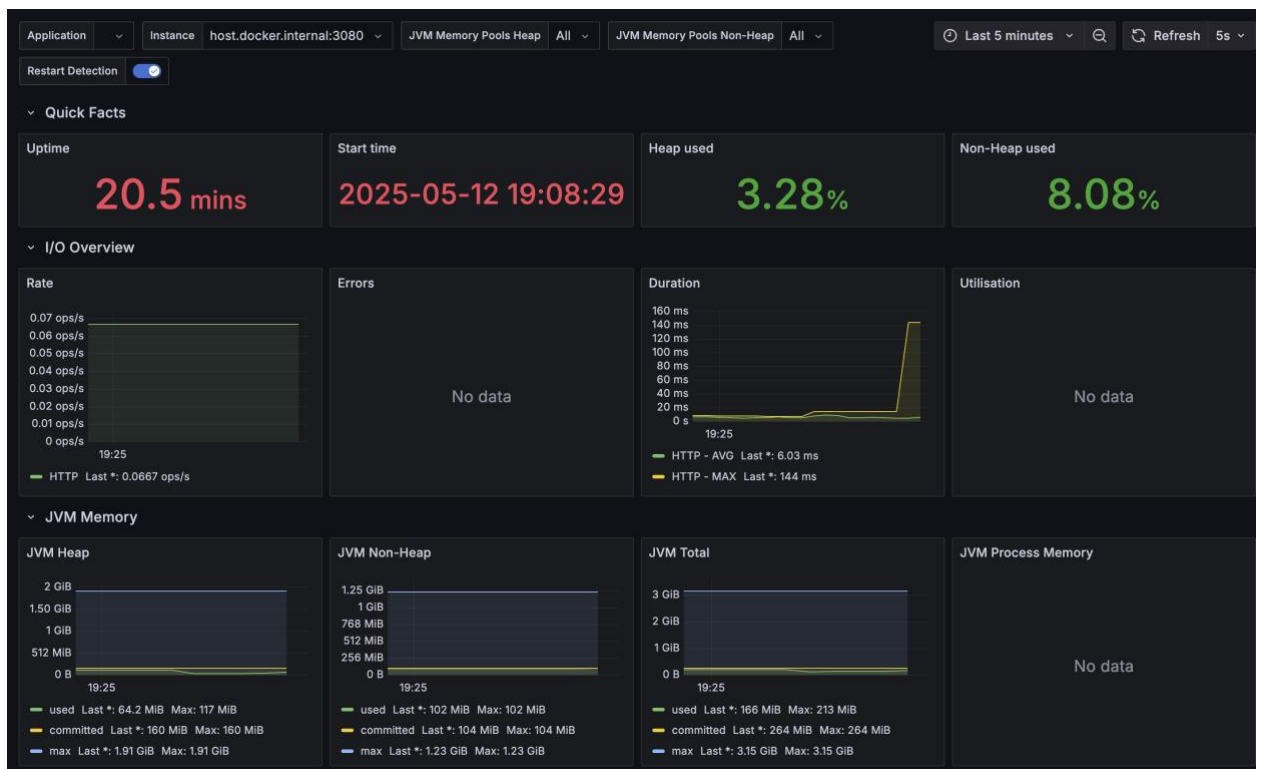


Рис. 3.8

3.3. Реализация прецедента «Опрос всех сервисов с заданной периодичностью на наличие доступности в текущий момент времени»

Реализована функция, которая запускается с определенной периодичностью. Внутри себя функция опрашивает сервис, если он активен, то проверяет связанные сервисы. В случае неактивности сервисов вызывает логику обработки инцидентов. Обновляет информацию в кеше о доступности сервисов. Обновление кеша нужно, чтобы клиентская часть приложения могла

в любой момент времени обратиться к этому кешу и получить актуальную информацию о состоянии сервисов.

3.3.1. Реализация алгоритма

Алгоритм проходит по всем сервисам, делает к ним запросы и формирует модель ответа с информацией об их доступности (рис.3.9).

Схема прецедента «Опрос всех сервисов с заданной периодичностью на наличие доступности в текущий момент времени»

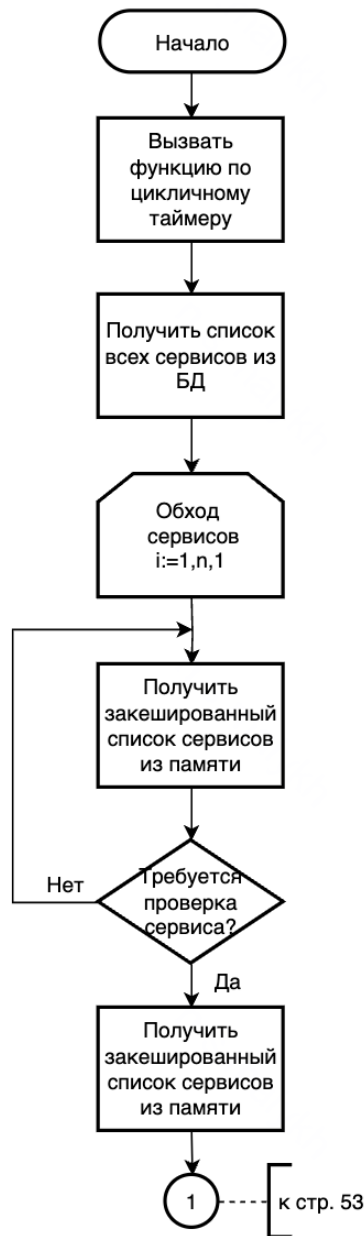
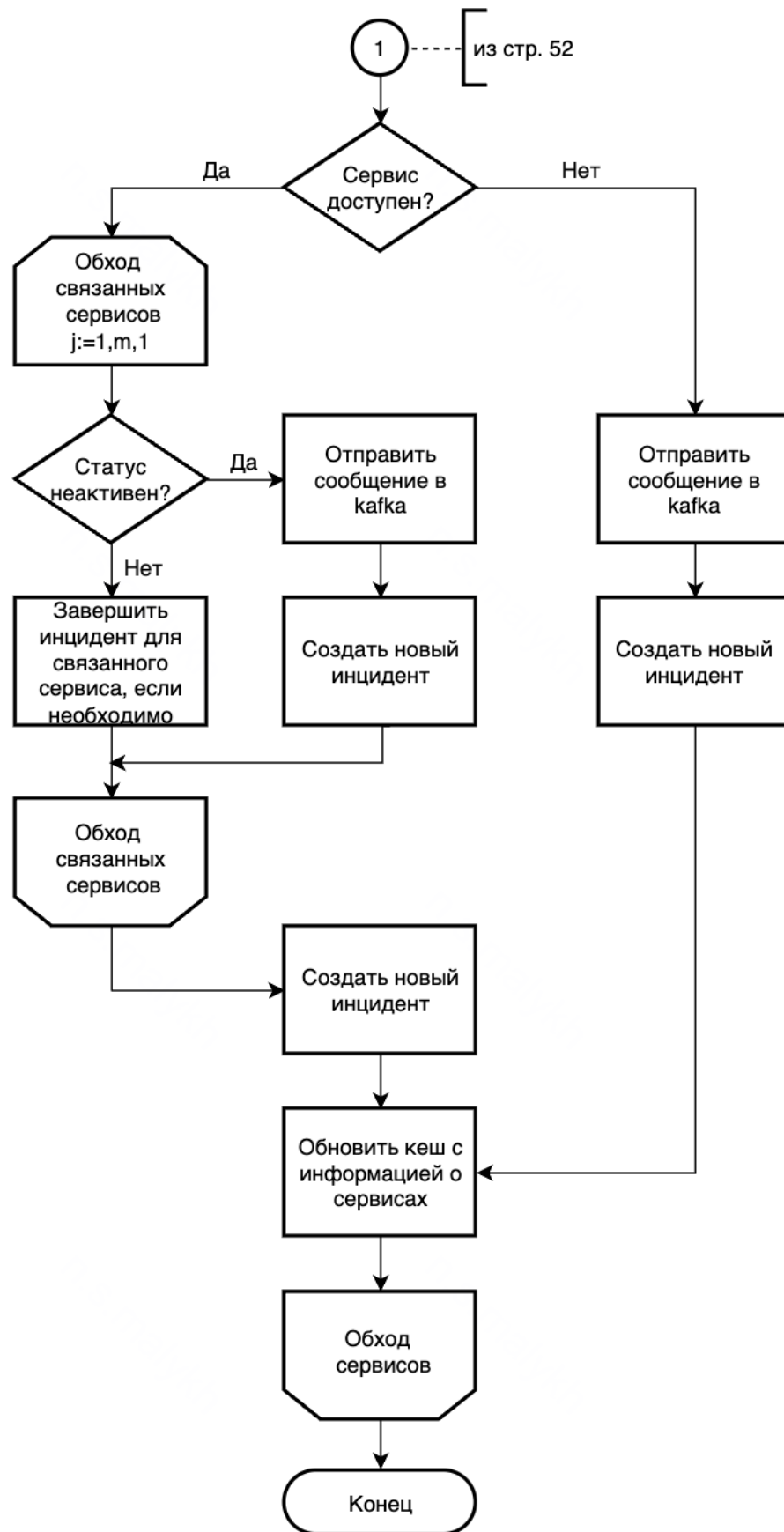


Рис. 3.9



Продолжение рис. 3.9

3.3.2. Реализация класса

Класс `EndpointService` отвечает за мониторинг состояния удаленных программных сервисов. Он выполняет периодическую проверку их доступности, обрабатывает ответы, регистрирует инциденты, сохраняет статусы и отправляет сообщения в `Kafka`. Является `Spring`-сервисом и запускается автоматически при старте приложения.

Сводка полей класса:

- `EndpointRepo` – репозиторий для взаимодействия с таблицей сервисов через `SpringDataJpa` интерфейс;
- `JwtTokenService` – сервис для получения роли текущего пользователя по `JWT`;
- `MessageSender` – сервис отправки сообщений в `Kafka`;
- `IncidentProcessService` – сервис обработки инцидентов;
- `RestClient` – `HTTP`-клиент для выполнения `REST`-запросов к другим сервисам;
- `EndpointWithTimeDto` – синглтон для реализации кеша в памяти;
- `String GET_TOKEN` – `url`-путь к ресурсу получения токена;
- `String CHECK_STATUS` – `url`-путь к ресурсу проверки доступности сервиса.

Сводка методов класса:

- `postConstruct()` – инициализирует кеш внутри памяти при старте приложения, чтобы не обращаться в БД при каждом запросе к этому сервису;
- `checkAllEndpoints()` – периодически запускает опрос всех сервисов, соблюдая ограничения по времени;
- `checkEndpointByUrl(String url)` – выполняет точечную проверку сервиса по его `url`;
- `performEndpointCheck(Endpoint endpoint)` – проверяет статус сервиса. Обновляет кеш и создает инциденты при сбоях в сервисах;

- `buildEndpointStatusDto()` – формирует `data-transfer-object(dto)` при успешной проверке, отправляет сообщения в Kafka, если сервис недоступен;
- `buildNoConnectionDto()` – формирует `dto` для случая, если сервиса не отвечает. Создает инцидент;
- `sendNoitifications(String message)` – отправляет сообщения через сервис отправки в Kafka;
- `checkServiceAvailability(String url, String token)` – отправляет GET-запрос с токеном авторизации на сервис, получает в ответе `dto`;
- `checkEndpointTimer(String username)` – определяет, можно ли снова проверять сервис, исходя из его интервала времени;
- `getSavedData()` – возвращает список статусов сервисов, доступных текущему пользователю. Скрывает часть операций для роли «user»;
- `formatNow()` – возвращает текущую дату и время в виде строки;
- `formatInstant(Instant instant)` – преобразует `Instant` в строку для отображения.

3.3.3. Описание контрольного примера

Клиентская часть после опроса сервисов

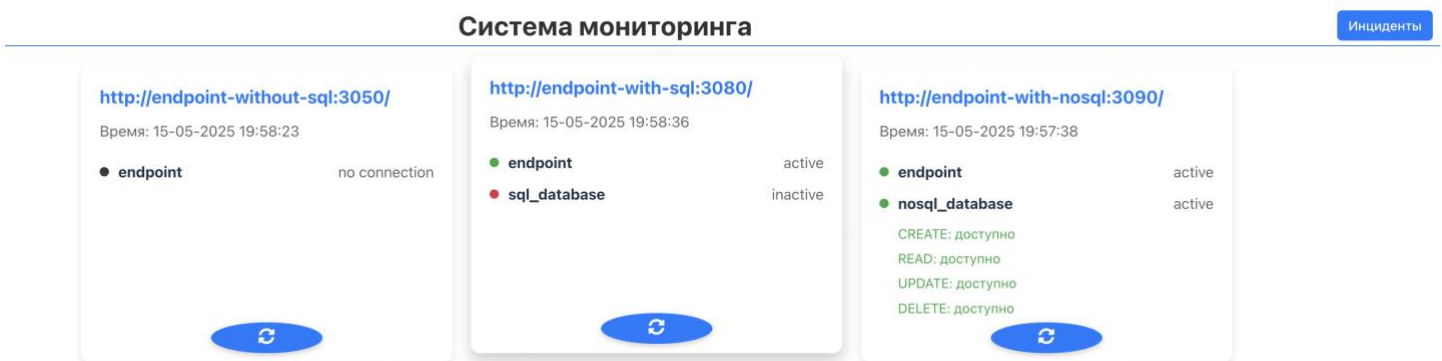


Рис. 3.10

3.4. Реализация прецедента «Точечный опрос сервиса на наличие доступности в текущий момент времени»

Запрос на серверную часть приложения, чтоб выполнить точечную проверку сервиса, чтобы не дожидаться таймера опроса всех сервисов. Логика опроса аналогична п.3.3, только опрашивается один сервис по уникальному

url, и в кеше обновляется информация только о нем. Полезно, если есть необходимость быстрой проверки сервиса.

3.4.1. Реализация алгоритма

Алгоритм (рис.3.11) запрашивает информацию о доступности только одного сервиса. Преимуществом над опросом всех сервисов является скорость обновления информации при необходимости вместо $O(n)$ будет $O(1)$. Это существенный прирост. Остальная логика опроса остается аналогичной, как и при опросе всех сервисов. Фиксация инцидентов аналогична.

Схема прецедента «Точечный опрос сервиса на наличие доступности в текущий момент времени»

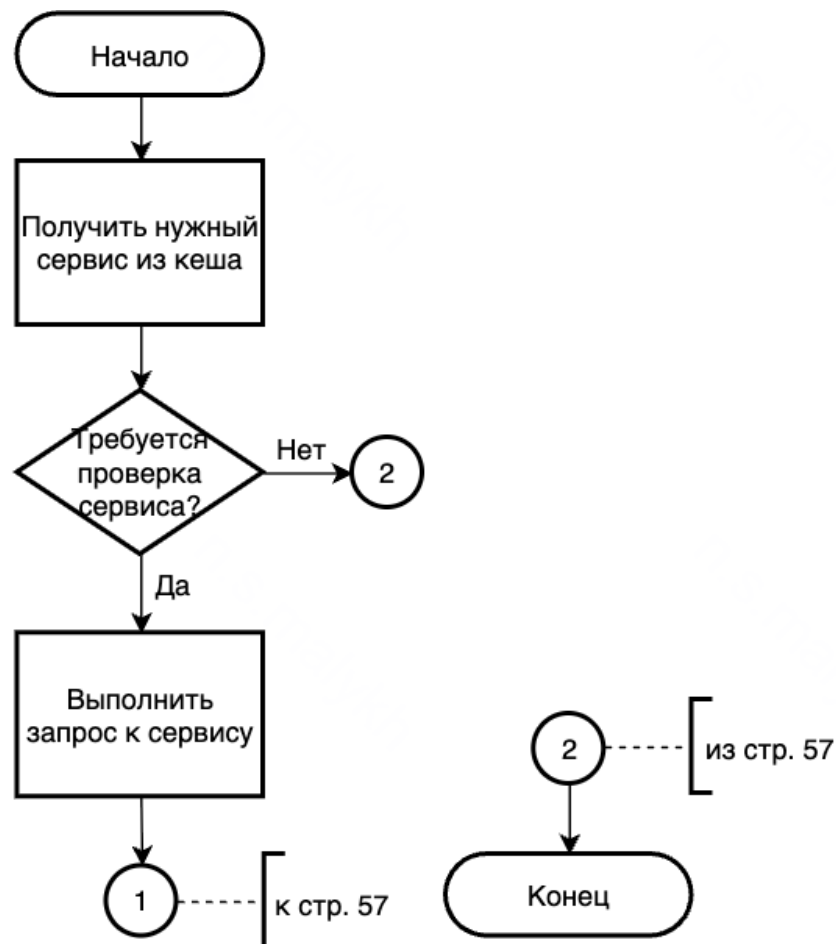
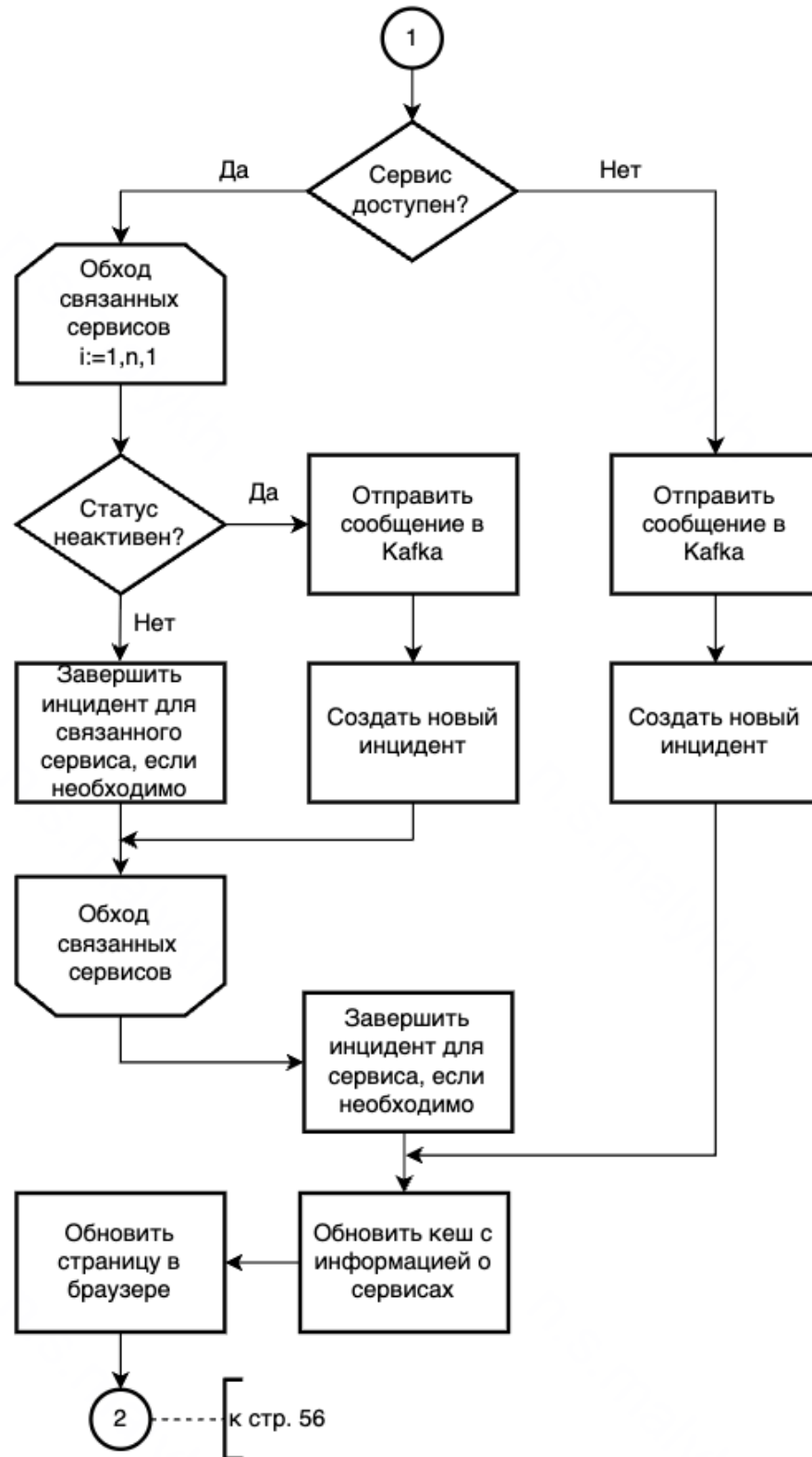


Рис. 3.11



Продолжение рис. 3.11

3.4.2. Реализация класса

Функционал реализован в классе, описанном в п.3.3.2.

3.4.3. Описание контрольного примера

Запрос в Postman для точечного обновления

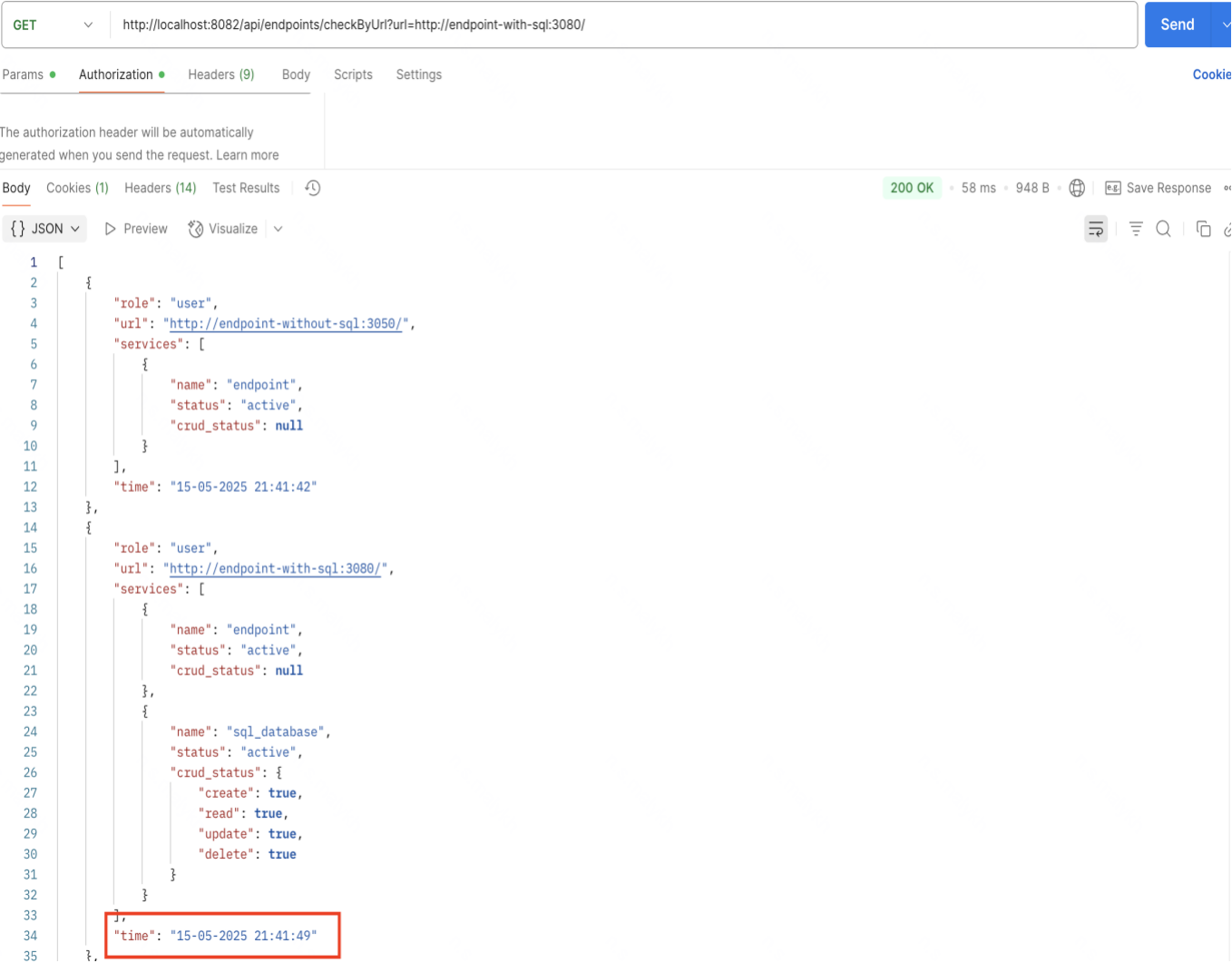


Рис. 3.12

В контрольном примере видим, что выполнен запрос на обновление информации о сервисе на порту 3080. По времени обновления можно увидеть, что информация о сервисе обновилась «только что», в отличие от сервиса на порту 3050, у которого последнее время обновления не изменилось.

3.5. Реализация прецедента «Создание инцидента в случае неактивности сервиса и завершение инцидента в случае, когда сервис вновь активен»

Предусловие: опрашиваются сервисы, каждый сервис может иметь связанные сервисы (например, БД). Статус «INACTIVE» значит, что сервис(или связанный сервис) в текущий момент времени не активен.

«ACTIVE» значит, что сервис(или связанный сервис) в текущий момент времени активен.

Сначала проверяем активен ли сервис. Если неактивен сам сервис, то проверяем, что в БД либо еще нет инцидента для этого сервиса либо если он есть, то имеет статус «ACTIVE», то есть он когда-то был, но уже завершился и просто хранится в БД для статистики. Если одно из этих условий выполнено, то сохраняем новый инцидент, заполняя дату начала инцидента и оставляя пустой дату окончания, также ставим статус «INACTIVE», так как сервис неактивен.

Когда получаем информацию о том, что сервис активен, то проверяем в цикле все связанные сервисы (БД и тому подобное). Если они неактивны, то по аналогии с основным сервисом сохраняем инцидент(то есть, если его еще не было или был, но уже в статусе «ACTIVE»), а если связанный сервис активен, то фиксируем инцидент, если это необходимо(то есть, если он есть в БД и находится в статусе «INACTIVE»). Также сразу считаем продолжительность инцидента и меняем статус. После проверки связанных сервисов фиксируем инцидент основного сервиса, если это необходимо по аналогии с фиксацией инцидентов на связанных сервисах.

3.5.1. Реализация алгоритма

Алгоритм на схеме реализует логику, описанную в п.3.5 (рис.3.13).

Схема прецедента «Создание инцидента в случае неактивности сервиса и завершение инцидента в случае, когда сервис вновь активен»

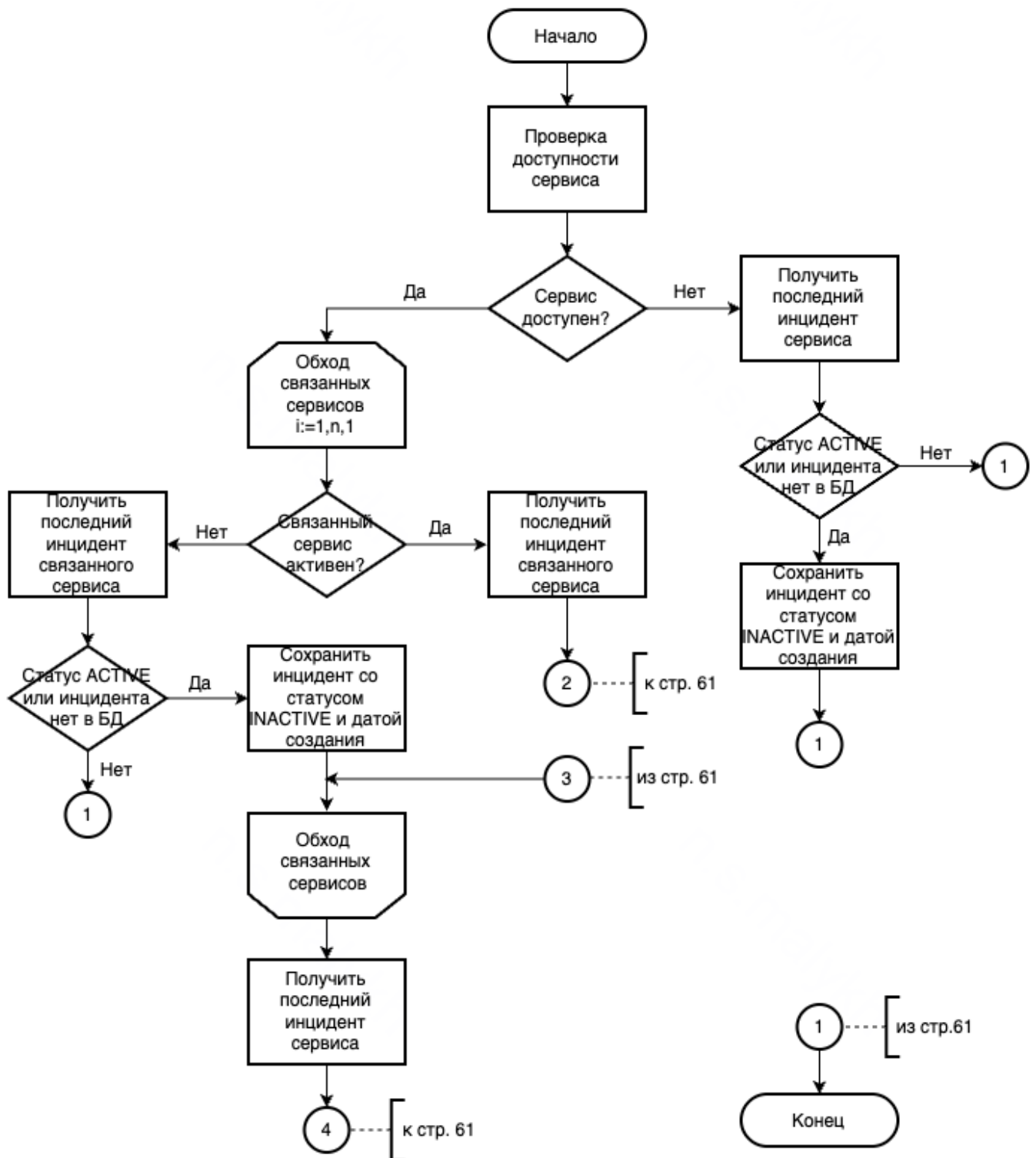
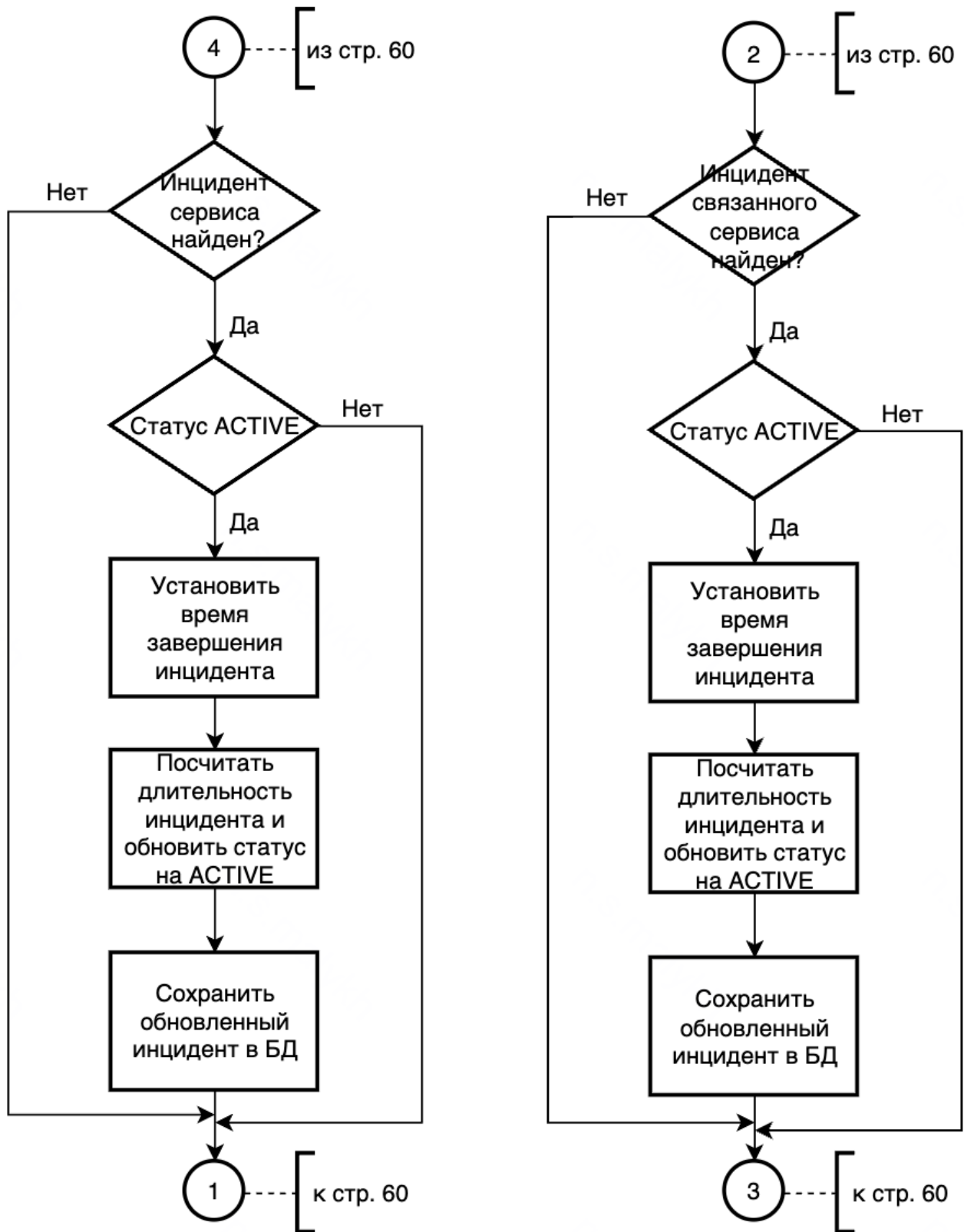


Рис. 3.13



Продолжение рис. 3.13

3.5.2. Реализация класса

Класс IncidentProcessService отвечает за обработку инцидентов сервисов. Является Spring-сервисом и запускается автоматически при старте приложения.

Содержит поле EndpointAvailabilityRepo – репозиторий, предназначенный для взаимодействия с таблицей «endpoint_available».

Сводка методов класса:

- createDBIncidentIfRequired(String serviceName) – создает инцидент в БД для связанных сервисов с основным сервисом;
- finishDBIncidentIfRequired(String serviceName) – фиксирует окончание инцидента в БД для связанных сервисов с основным сервисом;
- createServiceIncidentIfRequired(Endpoint endpoint) – создает инцидент в БД для основного сервиса;
- finishServiceIncidentIfRequired(Endpoint endpoint) – фиксирует окончание инцидента в БД для основного сервиса;
- checkIncidentAndSaveIfinactive(EndpointAvailability lastIncident) – выполняет сохранение инцидента, если он неактивен. При этом фиксирует время окончания и продолжительность инцидента.

3.5.3. Описание контрольного примера

Инциденты в форме клиентской части

Инциденты

Имя INACTIVE ДД.ММ.ГГГГ, --:-- ДД.ММ.ГГГГ, --:--

Поиск

Имя	Начало	Конец	Минут
Сервис №3 - С БД и с Redis	2025-05-14T14:11:28.984856		
Сервис №1 - Без БД	2025-05-14T14:07:49.328399		

Назад Страница 1 из 1 Вперед

Рис. 3.14

Завершенные инциденты в форме клиентской части

Имя

ACTIVE

ДД.ММ.ГГГГ, --:--

ДД.ММ.ГГГГ, --:--

Поиск

Имя	Начало	Конец	Минут
Сервис №3 - С БД и с Redis	2025-05-14T14:11:28.984856	2025-05-14T14:18:09.066119	6
Сервис №1 - Без БД	2025-05-14T14:07:49.328399	2025-05-14T14:18:28.940013	10
Сервис №3 - С БД и с Redis	2025-05-09T09:38:30.720926	2025-05-09T09:50:10.695899	11
Сервис №2 - С БД и без Redis	2025-05-09T09:38:30.618962	2025-05-09T09:49:50.735975	11
Сервис №1 - Без БД	2025-05-09T09:37:50.911019	2025-05-09T09:49:10.652243	11

Назад

Страница 1 из 2

Вперёд

Рис. 3.15

3.6. Реализация прецедента «Получение детальной информации о сервисе»

Вызывается контроллер с клиентской части при нажатии на карточку сервиса для того, чтобы получить информацию о сервисе для заполнения карточки. Происходит обращение в БД для получения информации и формирования ответа. Происходит валидация данных, то есть если сервиса нет в БД, то будет выброшено исключение. Иначе же будет сформирована модель данных для возврата на клиентскую часть.

3.6.1. Реализация алгоритма

На схеме представлен алгоритм реализации прецедента (рис.3.16).

Схема прецедента «Получение детальной информации о сервисе»



Рис. 3.16

3.6.2. Реализация класса

Класс `EndpointInfoService` предназначен для получения информации о карточке сервиса и для обновления информации о сервисе и информации о его карточке.

Сводка полей класса:

- `EndpointRepo` – репозиторий для получения сущностей сервисов из БД;

- EndpointInfoRepo – репозиторий для получения сущностей карточек сервисов;
- EndpointWithTimeDto – объект-синглтон, через который реализован кеш.

Сводка полей метода:

- getEndpointInfo(String url) – метод для получения основной информации по сервису;
- updateEndpointInfo(EndpointUpdateRequest updateRequest, String url) – метод для обновления информации сервиса и его карточки, полученной из формы клиентской части.

3.6.3.Описание контрольного примера

Вызов метода getEndpointInfo через Postman

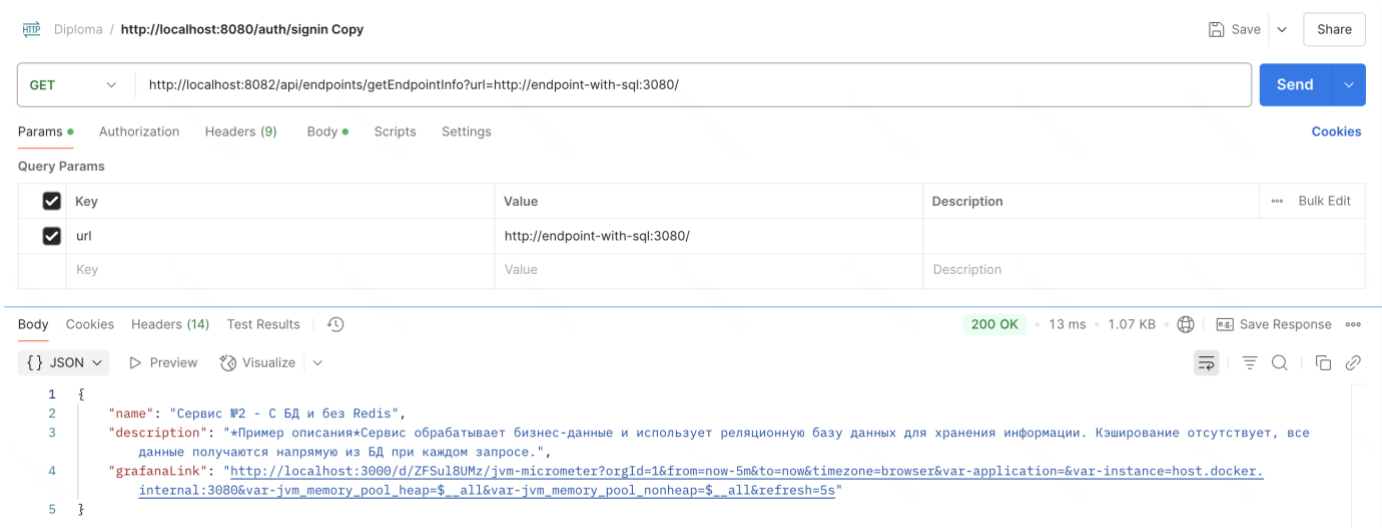


Рис. 3.17

3.7. Реализация прецедента «Обновление информации о сервисе»

Пользователем заполняется форма с данными на клиентской части и формируется запрос, который отправляется на серверную часть, где обрабатывается контроллером микросервиса. Все поля, которые не равны null в запросе участвуют в обновлении данных сервиса. Важным моментов является обновление периода и url. По причине того, что данные сервисов

кэшируются один раз и сохраняются в объекте синглтон, то помимо обновления в БД, нужно обновить и закэшированные данные (рис.3.18).

3.7.1.Реализация алгоритма

Схема прецедента «Обновление информации о сервисе»

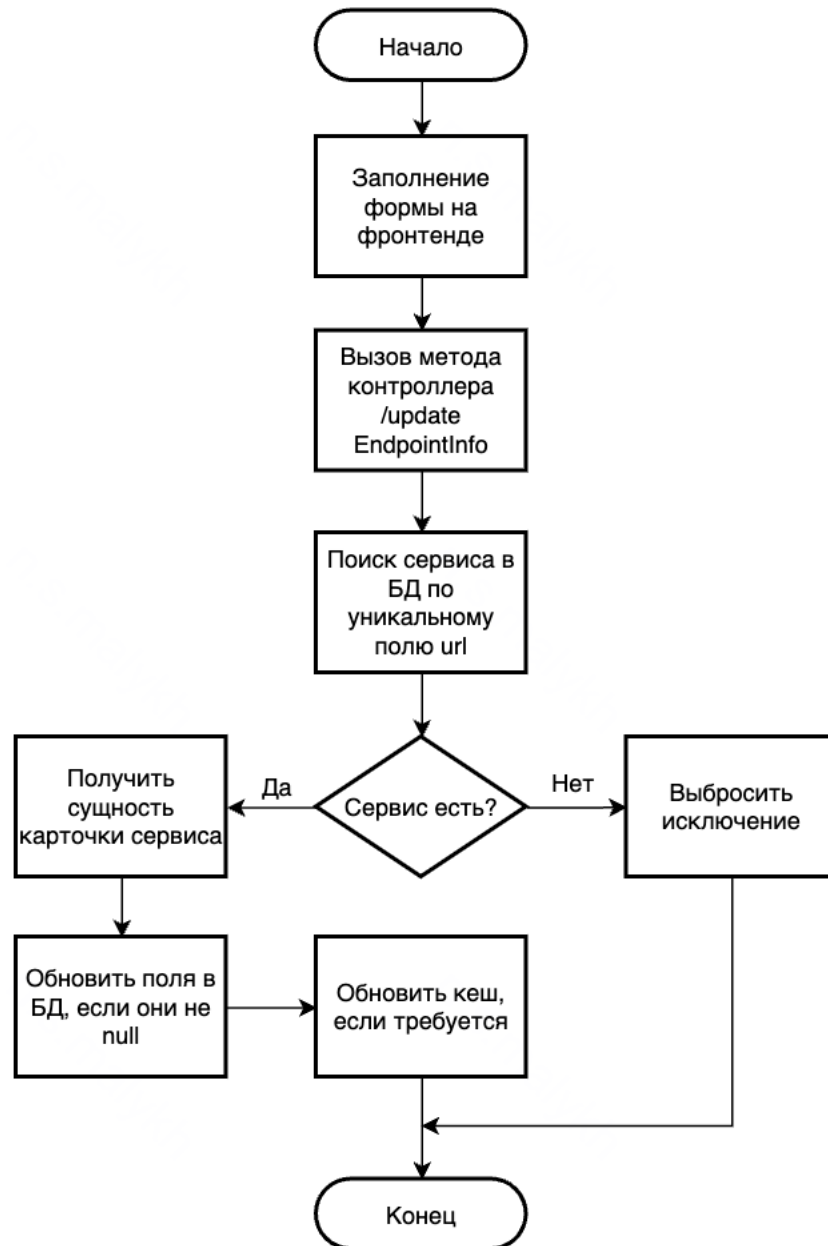


Рис. 3.18

3.7.2.Реализация класса

Тот же самый класс, что и в п.3.6.2.

3.7.3.Описание контрольного примера

Форма на клиентской части, в которой происходит заполнение полей для обновления информации о сервисе и его карточке

The screenshot shows a web form titled 'Сервис №2 - С БД и без Redis'. Below the title is a close button (X). The form has a section 'Обновить данные сервиса' containing five input fields: 'URL. Заполните, если необходимо', '200', 'Обновленный сервис', 'Обновленное описание сервиса', and 'Ссылка на Grafana. Заполните, если необходимо'. At the bottom of the form are two green buttons: 'Обновить данные сервиса' and 'Назад'.

Рис. 3.19

Результат обновления в карточке

The screenshot shows a card titled 'Обновленный сервис'. It contains the text 'Обновленное описание сервиса' and a blue link 'Перейти в Grafana'. There is a close button (X) in the top right corner and a settings gear icon in the bottom right corner.

Рис. 3.20

3.8. Реализация прецедента «Получение информации об инцидентах с функциональностью пагинации и гибкой фильтрации»

Пользователь на форме в клиентской части выбирает фильтры для получения списка инцидентов. Далее отправляется запрос на серверную часть, где происходит валидация входных данных. В формирование sql-запроса включаются те параметры фильтрации, которые были заданы. Серверная часть отдает на клиентскую часть данные не целиком, а постранично (реализована пагинация - по 5 элементов на странице).

3.8.1. Реализация алгоритма

Схема прецедента «Получение информации об инцидентах с функциональностью пагинации и гибкой фильтрации»

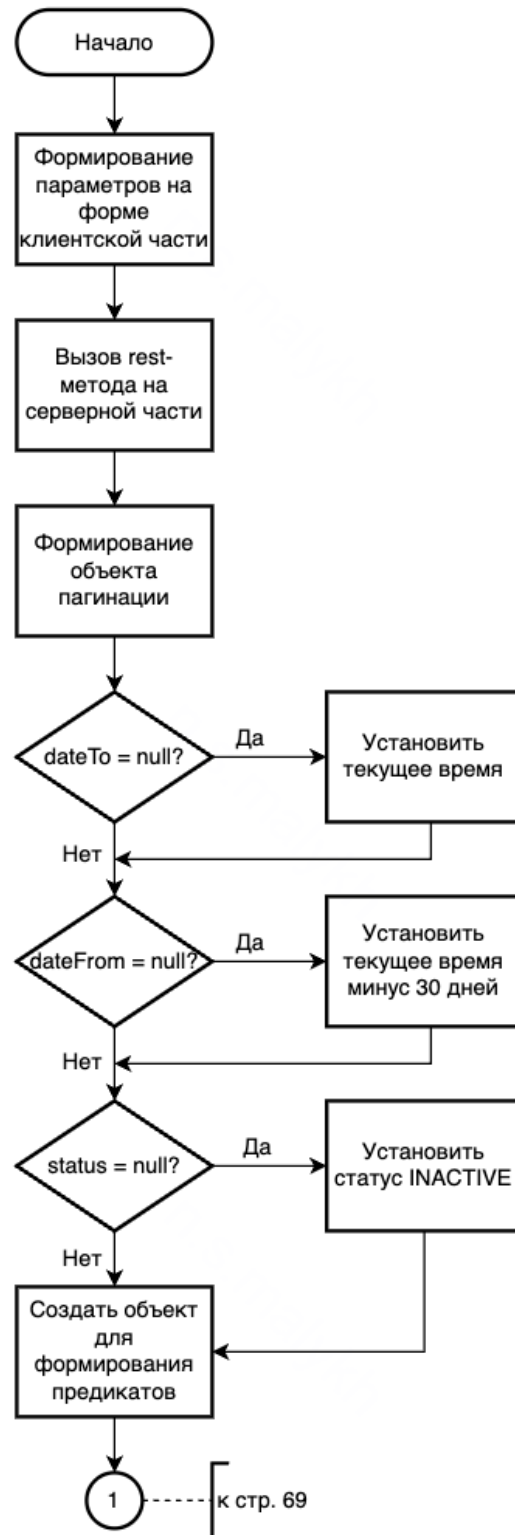
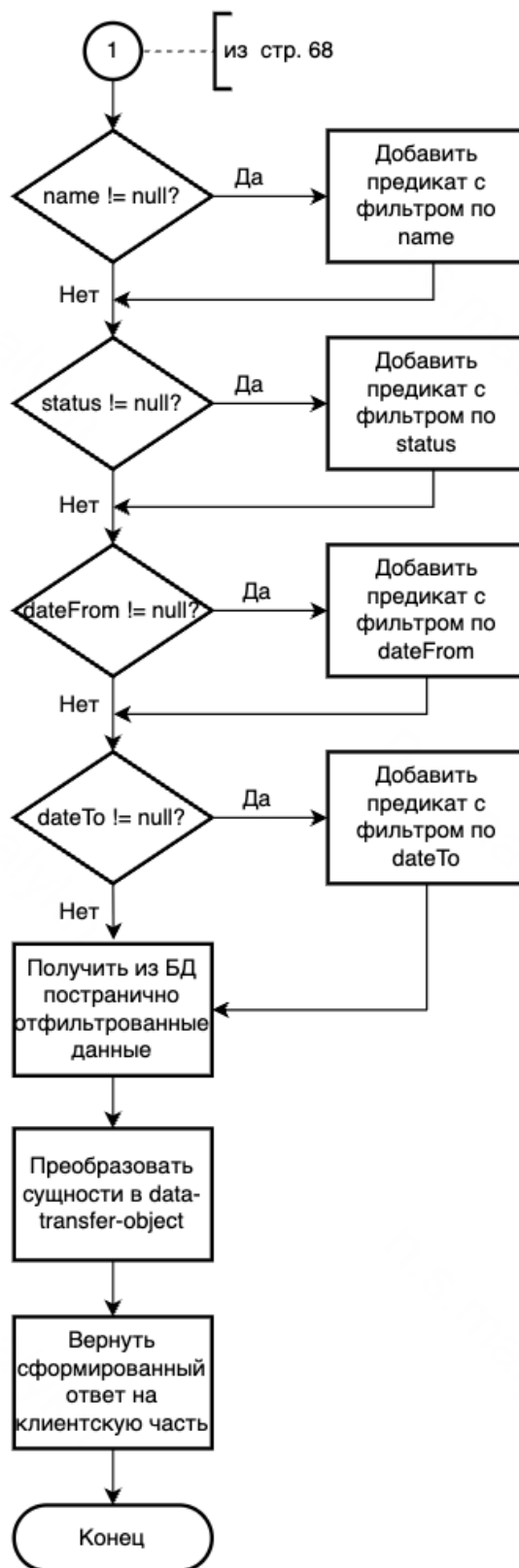


Рис. 3.21



Продолжение рис. 3.21

3.8.2. Реализация класса

Класс IncidentsProcessService.

Содержит поле `EndpointAvailabilityRepo` – репозиторий для взаимодействия с таблицей инцидентов в БД.

Сводка методов класса:

- `getIncidents(int page, int size, LocalDateTime dateFrom, LocalDateTime dateTo, EndpointStatus status, String name)` – формирует страницу результатов из БД заданного размера(по умолчанию 5 элементов) в отфильтрованном виде на основе переданных предикатов фильтрации;
- `hasName(String name)` – формирует предикат фильтрации по `name`;
- `hasStatus(Endpoint status)` – формирует предикат фильтрации по `status`;
- `hasDateFrom(LocalDateTime dateFrom)` – формирует предикат фильтрации по `dateFrom`;
- `hasDateTo(LocalDateTime dateTo)` – формирует предикат фильтрации по `dateTo`.

3.8.3. Описание контрольного примера

Запрос в Postman с одним фильтром и постраничный ответ (5 элементов на странице)

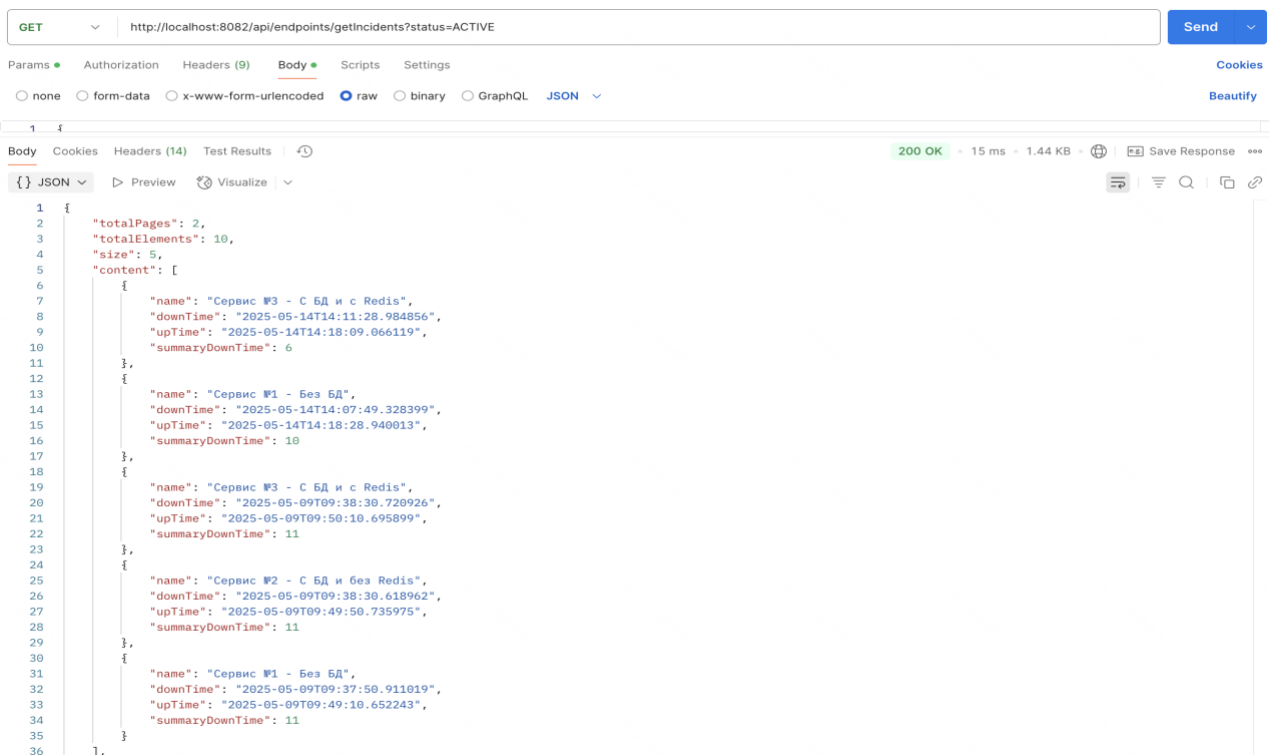


Рис. 3.22

3.9. Реализация прецедента «Отправка сообщений на почту и в телеграм-бота с логикой повторных попыток отправки и защитой от спама»

Сервис слушает топик в Kafka и при получении сообщения в партии считывает его. Далее формирует список получателей сообщения, подготавливает сообщения для отправки на почту и в сервис отправки в телеграм-бота. При успешной отправке включается таймер на 30 минут, чтобы отправить следующие сообщения не раньше этого времени, чтобы не было спама на почту и в телеграм-бота. Если же отправка неуспешная (4xx или 5xx коды ответа), то активируется логика ретраев (повторных попыток отправки сообщений с периодичностью 5 минут).

3.9.1. Реализация алгоритма

Схема прецедента «Отправка сообщений на почту и в телеграм-бота с логикой повторных попыток отправки и защитой от спама»

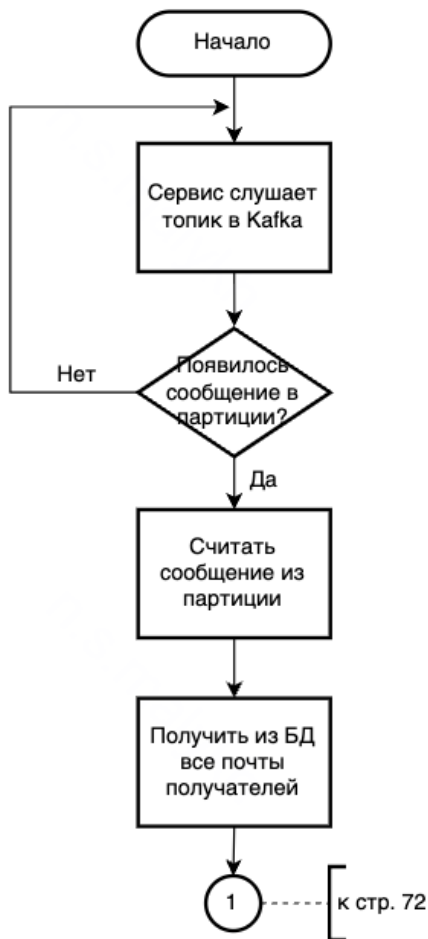
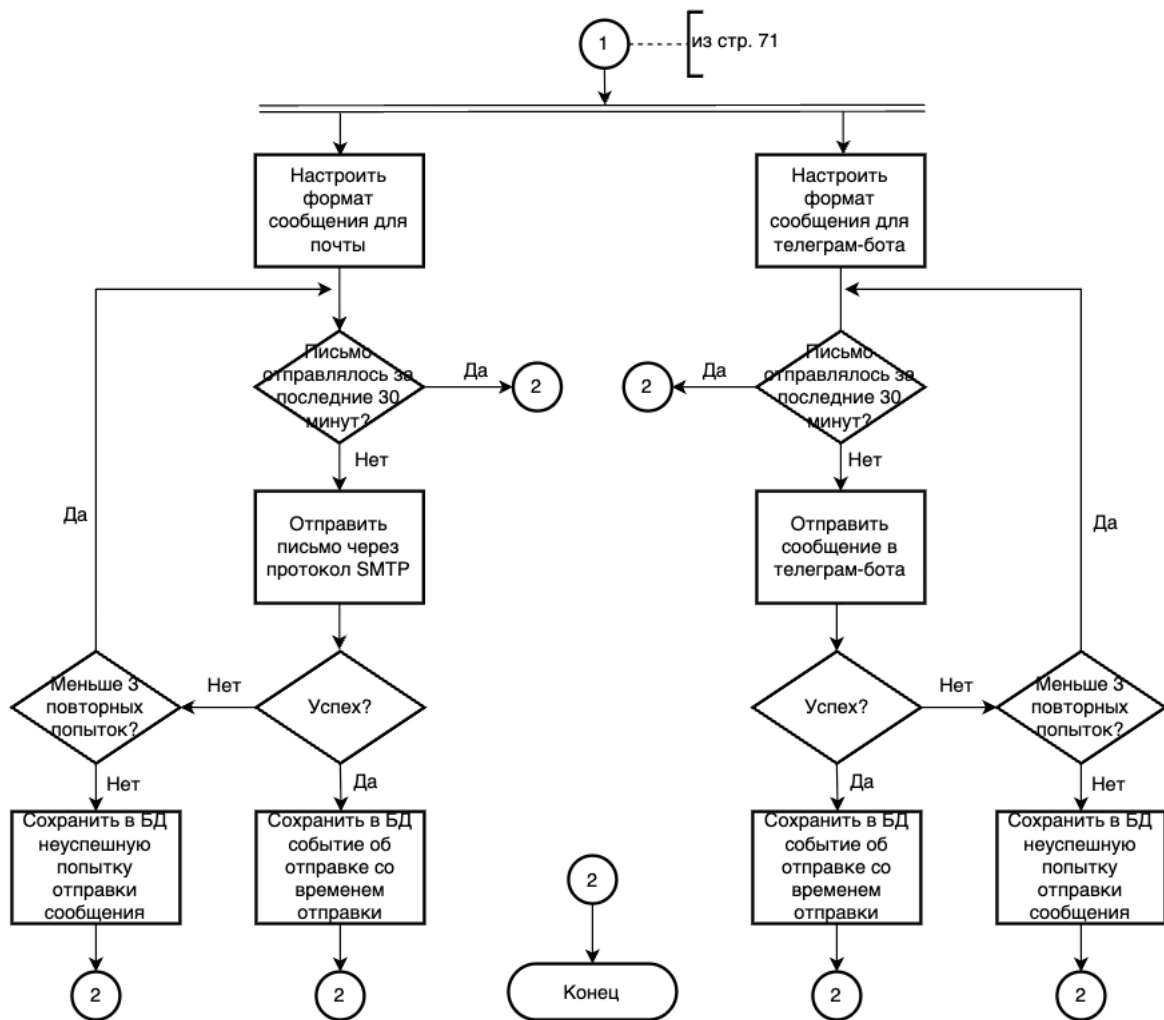


Рис. 3.23



Продолжение рис. 3.23

3.9.2. Реализация класса

Класс NotificationServiceImpl.

Сводка полей класса:

- EmailRepo – репозиторий для получения информации о получателях из таблицы БД;
- JavaMailSender – библиотечный класс из фреймворка Spring Framework для отправки сообщений на почту;

Сводка методов класса:

- getMessageFromKafka(String messageFromKafka) – помечен аннотацией @KafkaListener и является консьюмером, слушающим определенный топик на Kafka-сервере. Передает сообщение в другие функции.

- `sendMessageToEmail(String message)` – реализует логику отправки писем по протоколу SMTP с логикой повторной отправки и защиты от спама;
- `sendMessageToTG (String message)` – реализует логику отправки писем по протоколу SMTP с логикой повторной отправки и защиты от спама;

3.9.3. Описание контрольного примера

Полученное сообщение на почте



Рис. 3.24

Полученное сообщение в Telegram-боте

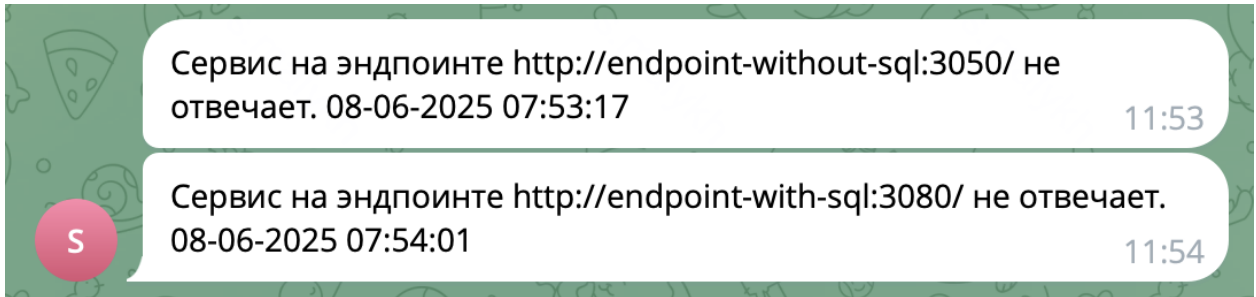


Рис. 3.25

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы была спроектирована и реализована система мониторинга сервисов, обеспечивающая автоматизированный сбор метрик, визуализацию состояния сервисов и оперативное оповещение об инцидентах. Разработанное решение построено на базе современного технологического стека, включающего Java 17, Spring Boot, Kafka, Prometheus, Grafana, PostgreSQL, Docker и React JS, что позволило создать масштабируемую и расширяемую микросервисную архитектуру.

Результатом работы стала полнофункциональная веб-система, включающая в себя:

- три микросервиса с индивидуальной реляционной базой данных, в том числе микросервис авторизации с использованием Redis;
- интеграцию Kafka между сервисом опросов и системой нотификаций;
- подключение к инструментам мониторинга Prometheus и Grafana;
- клиентскую часть, отображающую метрики и состояние сервисов;
- заглушки для тестирования мониторинга внешних сервисов.

Система позволяет накапливать информацию о произошедших инцидентах, анализировать метрики производительности в реальном времени и управлять параметрами опроса каждого сервиса. За счёт использования открытых и широко применяемых решений система остаётся гибкой и легко адаптируемой под нужды конкретной организации.

Практическая значимость работы заключается в возможности внедрения системы в IT-инфраструктуру небольших компаний и стартапов. Благодаря низкому порогу входа и невысокой стоимости эксплуатации она может использоваться как самостоятельный инструмент мониторинга, способный

минимизировать простой сервисов, избежать финансовых потерь и повысить уровень доверия пользователей.

Технико-экономическая эффективность внедрения системы заключается в снижении времени на обнаружение и устранение сбоев, уменьшении затрат на поддержку и предотвращении потерь, связанных с недоступностью ключевых сервисов. Открытость архитектуры и модульный подход способствуют быстрому масштабированию системы и её интеграции с уже существующими решениями.

Перспективы дальнейшего развития включают интеграцию с другими источниками данных и системами оповещения, а также разработку мобильной версии клиентского интерфейса. Также возможна реализация гибкой настройки политик опроса и оповещений в зависимости от критичности сервисов.

В целом, разработанное решение демонстрирует высокую степень применимости в условиях современных требований к устойчивости и надёжности программных продуктов и может стать основой для более масштабных систем управления отказоустойчивостью и мониторинга.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1) Бондарь А. И. Архитектура программных систем: микросервисы и распределённые приложения. М.: ДМК Пресс, 2021. – 320 с.
- 2) Шипицин С. Ю. Промышленные подходы к разработке ПО: DevOps, CI/CD и мониторинг. М.: Наука и Техника, 2021. – 354 с.
- 3) Самойлов К. Ю. Kafka: потоковая обработка данных и очереди сообщений. СПб.: Питер, 2022. – 336 с.
- 4) Turnbull J. The Prometheus Monitoring System. 2nd ed. Melbourne: Turnbull Press, 2021. – 242 p.
- 5) Иванов Д. А., Павлов П. С. Инструменты мониторинга и логирования распределённых систем. М.: ДМК Пресс, 2020. – 288 с.
- 6) Майоров А. С. Docker и контейнеризация приложений. М.: БХВ-Петербург, 2021. – 384 с.
- 7) Grafana Labs. Grafana Documentation. – Электронный ресурс. URL: <https://grafana.com/docs/> (дата обращения: 20.05.2025).
- 8) Гусев В. С., Артемьева И. В. Основы REST API: проектирование и реализация. СПб.: Питер, 2023. – 256 с.
- 9) Oracle. Java Platform, Standard Edition 17 Documentation. – Электронный ресурс. URL: <https://docs.oracle.com/en/java/javase/17/> (дата обращения: 18.05.2025).
- 10) Поляков С. В. PostgreSQL: практическое руководство. М.: ДМК Пресс, 2021. – 464 с.
- 11) Pivotal Software, Inc. Spring Boot Reference Documentation. – Электронный ресурс. URL: <https://docs.spring.io/spring-boot/docs/current/reference/html/> (дата обращения: 20.05.2025).
- 12) Черкасов П. Н. Основы разработки веб-приложений на React.js. СПб.: Питер, 2022. – 288 с.

ПРИЛОЖЕНИЕ 1

ТЕКСТЫ ПРОГРАММ

П.1.1 Код микросервиса авторизации

П.1.1.1 Код класса AuthController

```

@RestController
@RequiredArgsConstructor
@RequestMapping("/api/v1/")
public class AuthController {
    private final JwtService jwtService;
    private final AuthenticationManager authenticationManager;
    private final UserService userService;
    private final RefreshTokenService refreshTokenService;
    private final RedisTokenBlackList tokenBlacklist;

    @PostMapping("/login")
    public ResponseEntity<JwtResponseDto>
authenticateAndGetToken(@RequestBody SignInDto signInDto) {
        System.out.println("Зашел в функцию authenticateAndGetToken в
AuthMicroservice");

        Authentication authentication =
authenticationManager.authenticate(new
UsernamePasswordAuthenticationToken(signInDto.getUsername(),
signInDto.getPassword()));

        if (authentication.isAuthenticated()) {
            String accessToken =
jwtService.generateToken(signInDto.getUsername());
            String refreshToken =
refreshTokenService.createOrUpdateRefreshToken(signInDto.getUsername());
            System.out.println("accessToken: " + accessToken);
            System.out.println("refreshToken: " + refreshToken);
            return ResponseEntity.ok().body(JwtResponseDto.builder()
                .accessToken(accessToken)
                .refreshToken(refreshToken)
                .build());
        } else {
            throw new UsernameNotFoundException("invalid user request..!!");
        }
    }
}

```

```

@PostMapping("/logout")
public ResponseEntity<String> logout(HttpServletRequest request) {
    String authHeader = request.getHeader("Authorization");
    String token;

    if (authHeader != null && authHeader.startsWith("Bearer ")) {
        token = authHeader.substring(7);
        tokenBlacklist.addToBlacklist(token);
    }

    return ResponseEntity.ok("Logged out successfully");
}

@PostMapping("/register")
public ResponseEntity<User> registerUser(@RequestBody SignUpDto
signUpDto) {
    return ResponseEntity.ok().body(userService.save(signUpDto));
}

@PostMapping("/refreshToken")
public ResponseEntity<JwtResponseDto> refreshToken(@RequestBody
RefreshTokenRequestDto refreshTokenRequestDTO) {
    System.out.println(refreshTokenRequestDTO.toString());
    return
ResponseEntity.ok().body(refreshTokenService.findByToken(refreshTokenRequestD
TO.getToken())
        .map(refreshTokenService::verifyExpiration)
        .map(RefreshToken::getUser)
        .map(userInfo -> {
            String accessToken =
jwtService.generateToken(userInfo.getUsername());
            return JwtResponseDto.builder()
                .accessToken(accessToken)

.refreshToken(refreshTokenRequestDTO.getToken()).build();
        }).orElseThrow(() -> new RuntimeException("Refresh Token is
not in DB...!!")));
}

@GetMapping("/getCurrentRole")

```

```

    public ResponseEntity<String> getCurrentRole() {
        System.out.println("Зашел в функцию getCurrentRole в
AuthMicroservice");
        List<String> roles = new ArrayList<>();
        Authentication auth =
SecurityContextHolder.getContext().getAuthentication();
        if (auth != null && auth.isAuthenticated()) {
            for (GrantedAuthority authority : auth.getAuthorities()) {
                roles.add(authority.getAuthority());
            }
        }
        return ResponseEntity.ok().body(roles.get(0));
    }
}

```

П.1.1.2 Код класса JwtService

```

@Service
public class JwtService {

    @Value("${jwt.secret-key}")
    public String SECRET;

    public String extractUsername(String token) {
        return extractClaim(token, Claims::getSubject);
    }

    public Date extractExpiration(String token) {
        return extractClaim(token, Claims::getExpiration);
    }

    public <T> T extractClaim(String token, Function<Claims, T>
claimsResolver) {
        final Claims claims = extractAllClaims(token);
        return claimsResolver.apply(claims);
    }

    private Claims extractAllClaims(String token) {
        return Jwts
            .parserBuilder()
            .setSigningKey(getSignKey())
            .setAllowedClockSkewSeconds(60)
            .build()
            .parseClaimsJws(token)

```

```

        .getBody();
    }

    private Boolean isTokenExpired(String token) {
        return extractExpiration(token).before(new Date());
    }

    public Boolean validateToken(String token, UserDetails userDetails) {
        final String username = extractUsername(token);
        return (username.equals(userDetails.getUsername()) &&
!isTokenExpired(token));
    }

    public String generateToken(String username) {
        Map<String, Object> claims = new HashMap<>();
        return createToken(claims, username);
    }

    private String createToken(Map<String, Object> claims, String username) {
        return Jwts.builder()
            .setClaims(claims)
            .setSubject(username)
            .setIssuedAt(new Date(System.currentTimeMillis()))
            .setExpiration(new Date(System.currentTimeMillis() + 1000 *
60 * 1)) // 1 min
            .signWith(getSignKey(), SignatureAlgorithm.HS256).compact();
    }

    private Key getSignKey() {
        byte[] keyBytes = Decoders.BASE64.decode(SECRET);
        return Keys.hmacShaKeyFor(keyBytes);
    }
}

```

П.1.1.3 Код класса RefreshTokenService

```

@Service
@RequiredArgsConstructor
public class RefreshTokenService {
    private final RefreshTokenRepository refreshTokenRepo;
    private final UserRepo userRepo;
}

```



```

public String createOrUpdateRefreshToken(String username) {
    Optional<User> userOptional = userRepo.findByUsername(username);
    if (userOptional.isPresent()) {
        User user = userOptional.get();

        // Проверяем, есть ли уже RefreshToken для данного пользователя
        Optional<RefreshToken> refreshTokenOptional =
refreshTokenRepo.findByUser(user);
        RefreshToken refreshToken;

        if (refreshTokenOptional.isPresent()) {
            // Если токен уже существует, обновляем его
            refreshToken = refreshTokenOptional.get();
            refreshTokenRepo.save(refreshToken);
            return refreshToken.getToken();
        } else {
            // Если токена нет, создаем новый
            refreshToken = RefreshToken.builder()
                .id(UUID.randomUUID())
                .user(user)
                .token(UUID.randomUUID().toString())
                .expiryDate(Instant.now().plusMillis(1000 * 60 * 30))
// 30 мин

                .build();
            refreshTokenRepo.save(refreshToken);
            return refreshToken.getToken();
        }
    } else {
        throw new UsernameNotFoundException("Такого юзера нет!");
    }
}

public Optional<RefreshToken> findByToken(String token) {
    return refreshTokenRepo.findByToken(token);
}

public RefreshToken verifyExpiration(RefreshToken token) {
    if (token.getExpiryDate().compareTo(Instant.now()) < 0) {
        refreshTokenRepo.delete(token);
    }
}

```

```

        throw new RuntimeException(token.getToken() + " Refresh token is
expired. Please make a new login...!");
    }
    return token;
}
}

```

П.1.1.4 Код класса UserService

```

@Service
@RequiredArgsConstructor
public class UserService {
    private final UserRepo userRepo;
    private final RoleRepo roleRepo;
    private final PasswordEncoder encoder;

    public User save(SignUpDto signUpDto) {
        Optional<User> userOptional =
userRepo.findByUsername(signUpDto.getUsername());
        if (userOptional.isEmpty()) {
            // Подставляем роль по умолчанию
            String roleName = signUpDto.getRole();
            if (roleName == null || roleName.isBlank()) {
                roleName = "user";
            }

            Optional<Role> roleOptional = roleRepo.findByName(roleName);
            Role role;

            if (roleOptional.isPresent()) {
                role = roleOptional.get();
            } else {
                role = Role.builder()
                    .name(roleName)
                    .build();
                roleRepo.save(role);
            }

            User user = User.builder()
                .id(UUID.randomUUID())
                .username(signUpDto.getUsername())
                .password(encoder.encode(signUpDto.getPassword()))
                .role(role)

```

```

        .build();
        userRepo.save(user);
        return user;

    } else {
        throw new RuntimeException("Такой юзер уже есть!");
    }
}
}

```

П.1.2 Код микросервиса опроса программных сервисов

П.1.2.1 Код класса EndpointController

```

@RestController
@RequestMapping("/api/endpoints")
@RequiredArgsConstructor
public class EndpointController {
    private final Logger LOGGER =
        LoggerFactory.getLogger(EndpointController.class);
    private final EndpointService endpointService;
    private final EndpointInfoService endpointInfoService;
    private final JwtTokenService jwtTokenService;
    private final IncidentProcessService incidentProcessService;

    @GetMapping("/check")
    public ResponseEntity<List<OutputDataDto>> getCachedInfoForEndpoints() {
        List<OutputDataDto> list = endpointService.getSavedData();
        LOGGER.info("LIST: " + list.toString());

        String currentRole = WorkWithAuth.getCurrentRole(jwtTokenService);
        LOGGER.info("CURRENT ROLE: " + currentRole);

        List<OutputDataDto> outputList = new ArrayList<>();
        String formattedRole = currentRole.split("_")[1];
        if (!formattedRole.equals("admin")) {
            for (OutputDataDto output : list) {
                if (output.getRole().equals(formattedRole)) {
                    outputList.add(output);
                }
            }
        } else {
            outputList = list;
        }
    }
}

```

```

    }

    LOGGER.info("OUTPUT LIST: " + outputList);

    return ResponseEntity.ok(outputList);
}

@GetMapping("/checkByUrl")
public ResponseEntity<List<OutputDataDto>>
checkEndpointByUrl(@RequestParam("url") String url) {
    LOGGER.info("checkEndpointByUrl БИЗБАХ");
    LOGGER.info("url: " + url);
    endpointService.checkEndpointByUrl(url);

    List<OutputDataDto> outputList =
getCachedInfoForEndpoints().getBody();
    LOGGER.info("list: " + outputList);
    return ResponseEntity.ok(outputList);
}

@GetMapping("/getEndpointInfo")
public ResponseEntity<EndpointInfoResponse>
getEndpointInfo(@RequestParam("url") String url) {
    LOGGER.info("getEndpointInfo БИЗБАХ");
    EndpointInfoResponse endpointInfoResponse =
endpointInfoService.getEndpointInfo(url);
    LOGGER.info("endpointInfoResponse: " +
endpointInfoResponse.toString());
    return ResponseEntity.ok(endpointInfoResponse);
}

@PostMapping("/updateEndpointInfo")
public void updateEndpointInfo(
    @RequestBody EndpointUpdateRequest endpointUpdateRequest,
    @RequestParam("url") String url
) {
    endpointInfoService.updateEndpointInfo(endpointUpdateRequest, url);
}

@GetMapping("/getIncidents")
public ResponseEntity<Page<IncidentsResponse>> getIncidents(

```

```

        @RequestParam(name = "dateFrom", required = false)
        @DateTimeFormat(iso = DateTimeFormat.ISO.DATE_TIME) LocalDateTime dateFrom,
        @RequestParam(name = "dateTo", required = false)
        @DateTimeFormat(iso = DateTimeFormat.ISO.DATE_TIME) LocalDateTime dateTo,
        @RequestParam(name = "status", required = false) EndpointStatus
        status,

        @RequestParam(name = "name", required = false) String name,
        @RequestParam(name = "page", defaultValue = "0") int page,
        @RequestParam(name = "size", defaultValue = "5") int size
    ) {
        return ResponseEntity.ok(incidentProcessService.getIncidents(page,
        size, dateFrom, dateTo, status, name));
    }
}

```

П.1.2.2 Код класса EndpointInfoService

```

@Service
@RequiredArgsConstructor
public class EndpointInfoService {
    private final EndpointRepo endpointRepo;
    private final EndpointInfoRepo endpointInfoRepo;
    private final EndpointWithTimeDto endpointWithTimeDto =
        EndpointWithTimeDto.getInstance();

    @Transactional(readOnly = true)
    public EndpointInfoResponse getEndpointInfo(String url) {
        Endpoint endpoint = endpointRepo.findEndpointByUrl(url)
            .orElseThrow(() -> new RuntimeException("Endpoint not
        found"));
        EndpointInfo endpointInfo = endpoint.getEndpointInfo();
        return new EndpointInfoResponse(
            endpointInfo.getName(),
            endpointInfo.getDescription(),
            endpointInfo.getGrafanaLink()
        );
    }

    @Transactional
    public void updateEndpointInfo(EndpointUpdateRequest updateRequest,
        String url) {
        Endpoint endpoint = endpointRepo.findEndpointByUrl(url)

```

```

        .orElseThrow(() -> new RuntimeException("Endpoint not
found"));
        EndpointInfo endpointInfo = endpoint.getEndpointInfo();

        if (updateRequest.getUrl() != null) {
            endpoint.setUrl(updateRequest.getUrl());
            Map<String, TimeDto> timeObj = endpointWithTimeDto.getTimeObj();

            timeObj.get(endpoint.getUsername()).getEndpoint().setUrl(updateRequest.getUrl());
        }
        if (updateRequest.getPeriod() != null) {
            endpoint.setPeriod(updateRequest.getPeriod());
            Map<String, TimeDto> timeObj = endpointWithTimeDto.getTimeObj();

            timeObj.get(endpoint.getUsername()).setTimePeriod(Duration.ofSeconds(updateRequest.getPeriod()));
        }
        if (updateRequest.getName() != null)
            endpointInfo.setName(updateRequest.getName());
        if (updateRequest.getDescription() != null)
            endpointInfo.setDescription(updateRequest.getDescription());
        if (updateRequest.getGrafanaLink() != null)
            endpointInfo.setGrafanaLink(updateRequest.getGrafanaLink());
        endpointInfoRepo.save(endpointInfo);
    }
}

```

П.1.2.3 Код класса EndpointServiceImpl

```

@Service
@RequiredArgsConstructor
public class EndpointServiceImpl implements EndpointService {
    private static final Logger LOGGER =
        LoggerFactory.getLogger(EndpointServiceImpl.class);

    private final EndpointRepo endpointRepo;
    private final JwtTokenService jwtTokenService;
    private final MessageSender messageSender;
    private final IncidentProcessService incidentProcessService;
    private final RestClient restClient = RestClient.create();
    private final EndpointWithTimeDto endpointWithTimeDto =
        EndpointWithTimeDto.getInstance();
}

```

```

@Value("${url.get_token.key}")
private String GET_TOKEN;

@Value("${url.check_status.key}")
private String CHECK_STATUS;

@PostConstruct
public void postConstruct() {
    endpointWithTimeDto.init(endpointRepo);
}

@Override
@Scheduled(fixedRate = 20000)
public void checkAllEndpoints() {
    LOGGER.info("ВЫЗВАНА ФУНКЦИЯ checkAllEndpoints()");
    List<Endpoint> endpoints = endpointRepo.findAll();
    LOGGER.info("Текущее состояние времени: {}",
endpointWithTimeDto.getTimeObj());
    LOGGER.info("Эндпоинты: {}", endpoints);

    for (Endpoint endpoint : endpoints) {
        if (!checkEndpointTimer(endpoint.getUsername())) {
            LOGGER.info("Пропуск проверки по таймеру для: {}",
endpoint.getUsername());
            continue;
        }

        LOGGER.info("Проверка эндпоинта: {}", endpoint.getUrl());
        performEndpointCheck(endpoint);
    }
}

@Override
public void checkEndpointByUrl(String url) {
    Endpoint endpoint = endpointRepo.findEndpointByUrl(url)
        .orElseThrow(() -> new IllegalStateException("Эндпоинт не
найден"));

    performEndpointCheck(endpoint);
}

```

```

        Map<String, TimeDto> timeMap = endpointWithTimeDto.getTimeObj();
        timeMap.get(endpoint.getUsername()).setLastVisit(Instant.now());
    }

    private void performEndpointCheck(Endpoint endpoint) {
        String formattedTime = formatNow();
        LoginEndpointDto loginDto = new
LoginEndpointDto(endpoint.getUsername(), endpoint.getPassword());

        // подумать и переработать проверку доступности
        try {
            TokenDto tokenDto = restClient.post()
                .uri(endpoint.getUrl() + GET_TOKEN)
                .header("Content-Type", "application/json")
                .body(loginDto)
                .retrieve()
                .body(TokenDto.class);

            String token = Optional.ofNullable(tokenDto)
                .map(TokenDto::getToken)
                .orElseThrow(() -> new IllegalStateException("Token is
null"));

            AuthResponse authResponse =
checkServiceAvailability(endpoint.getUrl(), token);
            EndpointStatusDto statusDto = buildEndpointStatusDto(endpoint,
authResponse, formattedTime);

            incidentProcessService.finishServiceIncidentIfRequired(endpoint);

            endpointWithTimeDto.updateMap(endpoint.getUsername(), statusDto);
        } catch (RestClientException | IllegalStateException e) {
            LOGGER.warn("Ошибка при проверке эндпоинта {}: {}",
endpoint.getUrl(), e.getMessage());
            EndpointStatusDto statusDto = buildNoConnectionDto(endpoint,
formattedTime);
            LOGGER.info("Статус эндпоинта после buildNoConnectionDto: {}",
statusDto);
            endpointWithTimeDto.updateMap(endpoint.getUsername(), statusDto);
        }
    }
}

```



```

        private EndpointStatusDto buildEndpointStatusDto(Endpoint endpoint,
AuthResponse response, String time) {
            EndpointStatusDto statusDto = new EndpointStatusDto(
                endpoint.getRole().getName(), endpoint.getUrl(), new
ArrayList<>()
            );

            for (ServiceDto service : response.getServices()) {
                if ("inactive".equals(service.getStatus())) {
                    String message = String.format(
                        "Сервис %s не работает на эндпоинте %s. %s",
service.getName(), endpoint.getUrl(), time
                    );
                    sendNotifications(message);

incidentProcessService.createDBIncidentIfRequired(service.getName());
                } else
incidentProcessService.finishDBIncidentIfRequired(service.getName());
                statusDto.getServices().add(service);
            }

            return statusDto;
        }

        private EndpointStatusDto buildNoConnectionDto(Endpoint endpoint, String
time) {
            String message = String.format("Сервис на эндпоинте %s не отвечает.
%s", endpoint.getUrl(), time);
            sendNotifications(message);
            incidentProcessService.createServiceIncidentIfRequired(endpoint);
            ServiceDto service = new ServiceDto(
                "endpoint", "no connection",
                new CrudStatusDto(false, false, false, false)
            );
            return new EndpointStatusDto(endpoint.getRole().getName(),
endpoint.getUrl(), List.of(service));
        }

        private void sendNotifications(String message) {
            MessageDto dto = MessageDto.builder().message(message).build();

```

```

        messageSender.sendMessageToNotificationCenter(dto);
    }

    private AuthResponse checkServiceAvailability(String url, String token) {
        return restClient.get()
            .uri(url + CHECK_STATUS)
            .header("token", token)
            .retrieve()
            .body(AuthResponse.class);
    }

    private boolean checkEndpointTimer(String username) {
        Map<String, TimeDto> timeMap = endpointWithTimeDto.getTimeObj();

        timeMap.computeIfAbsent(username, user ->
            endpointRepo.findEndpointByUsername(user)
                .map(e -> new TimeDto(new EndpointStatusDto(), Instant.now(),
                    e.getPeriod()))
                .orElse(null));

        TimeDto timeDto = timeMap.get(username);
        if (timeDto == null) return false;

        if
            (timeDto.getLastVisit().plus(timeDto.getTimePeriod()).isBefore(Instant.now())
        ) {
            timeDto.setLastVisit(Instant.now());
            return true;
        }
        return false;
    }

    @Override
    public List<OutputDataDto> getSavedData() {
        List<OutputDataDto> result = new ArrayList<>();
        Map<String, TimeDto> timeMap = endpointWithTimeDto.getTimeObj();
        String currentRole =
            WorkWithAuth.getCurrentRole(jwtTokenService).split("_")[1];

        for (TimeDto timeDto : timeMap.values()) {
            OutputDataDto output = new OutputDataDto();

```

```

        output.setUrl(timeDto.getEndpoint().getUrl());
        output.setRole(timeDto.getEndpoint().getRole());
        output.setServices(timeDto.getEndpoint().getServices());
        output.setTime(formatInstant(timeDto.getLastVisit()));
        if ("user".equals(currentRole) && output.getServices() != null) {
            for (ServiceDto service : output.getServices()) {
                if (service.getCrud_status() != null) {
                    service.getCrud_status().setCreate(false);
                    service.getCrud_status().setDelete(false);
                }
            }
        }
        result.add(output);
    }
    return result;
}

private String formatNow() {
    return DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm:ss")
        .format(LocalDateTime.now());
}

private String formatInstant(Instant instant) {
    return DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm:ss")
        .withZone(ZoneId.systemDefault())
        .format(instant);
}
}

```

П.1.2.4 Код класса IncidentProcessService

```

@Service
@RequiredArgsConstructor
public class IncidentProcessService {
    private final EndpointAvailabilityRepo endpointAvailabilityRepo;

    public void createDBIncidentIfRequired(String serviceName) {
        EndpointAvailability lastIncident = endpointAvailabilityRepo
            .findFirstByNameOrderByDownTimeDesc(serviceName)
            .orElse(null);

        //если инцидента нет или он когда то был, но уже завершился, то
        создать новый
    }
}

```

```

        if (lastIncident == null || lastIncident.getStatus() ==
EndpointStatus.ACTIVE) {
            endpointAvailabilityRepo.save(EndpointAvailability.builder()
                .id(UUID.randomUUID())
                .name(serviceName)
                .status(EndpointStatus.INACTIVE)
                .downTime(LocalDateTime.now())
                .build()
            );
        }
    }

    public void finishDBIncidentIfRequired(String serviceName) {
        EndpointAvailability lastIncident = endpointAvailabilityRepo
            .findFirstByNameOrderByDownTimeDesc(serviceName)
            .orElse(null);

        checkIncidentAndSaveIfInactive(lastIncident);
    }

    public void createServiceIncidentIfRequired(Endpoint endpoint) {
        EndpointAvailability lastIncident = endpointAvailabilityRepo
            .findFirstByEndpointIdOrderByDownTimeDesc(endpoint.getId())
            .orElse(null);

        if (lastIncident == null || lastIncident.getStatus() ==
EndpointStatus.ACTIVE) {
            endpointAvailabilityRepo.save(EndpointAvailability.builder()
                .id(UUID.randomUUID())
                .name(endpoint.getEndpointInfo().getName())
                .status(EndpointStatus.INACTIVE)
                .downTime(LocalDateTime.now())
                .endpoint(endpoint)
                .build()
            );
        }
    }

    public void finishServiceIncidentIfRequired(Endpoint endpoint) {
        EndpointAvailability lastIncident = endpointAvailabilityRepo
            .findFirstByEndpointIdOrderByDownTimeDesc(endpoint.getId())

```

```

        .orElse(null);
        checkIncidentAndSaveIfInactive(lastIncident);
    }

    private void checkIncidentAndSaveIfInactive(EndpointAvailability
lastIncident) {
        if (lastIncident != null && lastIncident.getStatus() ==
EndpointStatus.INACTIVE) {
            lastIncident.setUpTime(LocalDateTime.now());

lastIncident.setSummaryDownTime(Duration.between(lastIncident.getDownTime(),
LocalDateTime.now()).toMinutes());
            lastIncident.setStatus(EndpointStatus.ACTIVE);
            endpointAvailabilityRepo.save(lastIncident);
        }
    }

    public Page<IncidentsResponse> getIncidents(
        int page,
        int size,
        LocalDateTime dateFrom,
        LocalDateTime dateTo,
        EndpointStatus status,
        String name
    ) {
        System.out.println("Page: " + page + " Size: " + size + " DateFrom: "
+ dateFrom + " DateTo: " + dateTo + " Status: " + status + " Name: " + name);
        Pageable pageable = PageRequest.of(page, size,
Sort.by(Sort.Direction.DESC, "downTime"));

        if (dateTo == null) dateTo = LocalDateTime.now();
        if (dateFrom == null) dateFrom = LocalDateTime.now().minusDays(30);
        if (status == null) status = EndpointStatus.INACTIVE;

        Specification<EndpointAvailability> spec = Specification
            .where(IncidentProcessService.hasName(name))
            .and(IncidentProcessService.hasStatus(status))
            .and(IncidentProcessService.dateFrom(dateFrom))
            .and(IncidentProcessService.dateTo(dateTo));
    }

```

```

        Page<EndpointAvailability> incidents =
endpointAvailabilityRepo.findAll(spec, pageable);

        System.out.println("INCIDENTS: " + incidents.getContent());

        return incidents.map(IncidentsResponse::fromEntity);
    }

    public static Specification<EndpointAvailability> hasName(String name) {
        return (root, query, cb) -> name == null ? null :
            cb.like(cb.lower(root.get("name")), "%" + name.toLowerCase()
+ "%");
    }

    public static Specification<EndpointAvailability>
hasStatus(EndpointStatus status) {
        return (root, query, cb) -> status == null ? null :
            cb.equal(root.get("status"), status);
    }

    public static Specification<EndpointAvailability> dateFrom(LocalDateTime
dateFrom) {
        return (root, query, cb) -> dateFrom == null ? null :
            cb.greaterThanOrEqualTo(root.get("downTime"), dateFrom);
    }

    public static Specification<EndpointAvailability> dateTo(LocalDateTime
dateTo) {
        return (root, query, cb) -> dateTo == null ? null :
            cb.lessThanOrEqualTo(root.get("downTime"), dateTo);
    }
}

```

П.1.2.5 Код класса MessageSender

```

@Component
@RequiredArgsConstructor
public class MessageSender {
    private final KafkaProducerService kafkaProducerService;

    public void sendMessageToNotificationCenter(MessageDto messageDto) {
        kafkaProducerService.sendMessage("my-topic",
messageDto.getMessage());
    }
}

```

П.1.3 Код микросервиса нотификаций

П.1.3.1 Код класса NotificationService

```

@Service
@RequiredArgsConstructor
public class NotificationService {
    private final ObjectMapper objectMapper;
    private final Logger LOGGER =
        LoggerFactory.getLogger(NotificationService.class);

    private final
com.malykhnik.notification_microservice.repository.SpamDefenderRepo
spamDefenderRepo;
    private final EmailRepo emailRepo;
    private final JavaMailSender mailSender;
    @Value("${email.from}")
    String EMAIL_FROM;
    @Value("${url.tg.key}")
    String URL_TG;
    @Value("${url.tg_req.key}")
    String TG_REQUEST;

    @KafkaListener(topics = "my-topic", groupId = "my-group")
    public void getMessageFromKafka(String messageTxt) {
        MessageDto messageDto = null;
        boolean isRecievedAvailable = false;
        try {
            messageDto = objectMapper.readValue(messageTxt,
MessageDto.class);
            if (!checkIsMessageReceivedLastTime(messageDto.getUniqName())) {
                LOGGER.warn("Сообщение уже отправлялось ранее");
            } else isRecievedAvailable = true;
        } catch (Exception e) {
            LOGGER.error("Ошибка при получении сообщения из Kafka", e);
        }

        LOGGER.info("ВЫЗВАНА ФУНКЦИЯ sendMail ");
        LOGGER.info("Получил из Kafka: " + messageTxt);
        if (isRecievedAvailable) {
            SimpleMailMessage message =
createBaseMessage(messageDto.getMessage());
            processAndSendMessage(message);

            Optional<SpamDefender> spamDefender =
spamDefenderRepo.findByServiceName(messageDto.getUniqName());
            if (spamDefender.isPresent()) {
                spamDefender.get().setLastNotification(LocalDateTime.now());
                spamDefenderRepo.save(spamDefender.get());
            } else {
                spamDefenderRepo.save(new
SpamDefender(UUID.randomUUID(), messageDto.getUniqName(),
LocalDateTime.now()));
            }
        }

        private void processAndSendMessage(SimpleMailMessage message) {
            int maxRetries = 3;
            long delayMinutes = 5;

```

```

        ScheduledExecutorService scheduler =
Executors.newScheduledThreadPool(2);
        AtomicBoolean emailSuccess = new AtomicBoolean(false);
        AtomicBoolean tgSuccess = new AtomicBoolean(false);
        Runnable emailTask = new Runnable() {
            int attempt = 1;
            @Override
            public void run() {
                if (emailSuccess.get() || attempt > maxRetries) return;

                List<Email> emails = emailRepo.findAll();
                for (Email email : emails) {
                    message.setTo(email.getReceiver());
                    try {
                        LOGGER.info("ПОПЫТКА {}: Отправка email на {}",
attempt, email.getReceiver());
                        mailSender.send(message);
                        LOGGER.info("Письмо отправлено успешно");
                        emailSuccess.set(true);
                        return; // прекращаем дальнейшие попытки
                    } catch (Exception e) {
                        LOGGER.error("Ошибка при попытке {} отправки письма:
{}", attempt, e.getMessage());
                    }
                }

                if (attempt++ < maxRetries && !emailSuccess.get()) {
                    scheduler.schedule(this, delayMinutes, TimeUnit.MINUTES);
                }
            }
        };

        Runnable tgTask = new Runnable() {
            int attempt = 1;

            @Override
            public void run() {
                if (tgSuccess.get() || attempt > maxRetries) return;

                try {
                    LOGGER.info("ПОПЫТКА {}: Отправка сообщения в TG",
attempt);
                    WebClient.create(URL_TG).post()
                        .uri(TG_REQUEST)
                        .header("Content-Type", "text/plain")

                        .bodyValue(java.util.Objects.requireNonNull(message.getText()))
                        .retrieve()
                        .bodyToMono(String.class)
                        .doOnSuccess(response -> {
                            LOGGER.info("TG сообщение отправлено успешно:
{}", response);
                            tgSuccess.set(true);
                        })
                        .doOnError(error -> {
                            LOGGER.error("Ошибка при попытке {} отправки
в TG: {}", attempt, error.getMessage());
                            if (attempt++ < maxRetries &&
!tgSuccess.get()) {
                                scheduler.schedule(this,
delayMinutes, TimeUnit.MINUTES);

```



```

        }
    })
    .subscribe();
    } catch (Exception e) {
        LOGGER.error("Ошибка при формировании сообщения для TG:
{}", e.getMessage());
        if (attempt++ < maxRetries && !tgSuccess.get()) {
            scheduler.schedule(this, delayMinutes,
TimeUnit.MINUTES);
        }
    }
}

};

scheduler.execute(emailTask);
scheduler.execute(tgTask);
}

private SimpleMailMessage createBaseMessage(String messageTxt) {
    SimpleMailMessage message = new SimpleMailMessage();
    message.setFrom(EMAIL_FROM);
    message.setSubject("Отключение сервиса");
    message.setText(messageTxt);
    return message;
}

private Boolean checkIsMessageReceivedLastTime(String serviceName) {
    Optional<SpamDefender> spamDefenderOptional =
spamDefenderRepo.findByServiceName(serviceName);
    if (spamDefenderOptional.isPresent()) {
        SpamDefender spamDefender = spamDefenderOptional.get();
        return
!spamDefender.getLastNotification().isAfter(LocalDateTime.now().minusMinutes(
30));
    }
    return true;
}
}

```

ПРИЛОЖЕНИЕ 2

РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

П.2.1. Назначение системы

Назначением системы мониторинга программных сервисов является обеспечение автоматизации сбора метрик и мониторинга сервисов, а также оповещения пользователей об инцидентах.

П.2.2. Условия применения системы

Клиентская часть приложения доступна для компьютеров с установленным веб-браузером. Для работы серверной части приложения необходим сервер на ОС GNU/Linux с установленным ПО Docker и Docker Compose. Минимально допустимый для работы системы сервер должен иметь процессор с 4 физическими ядрами на частоте не менее 2.8 ГГц, 8 Гб оперативной памяти и жёстким диском объёмом не менее 100 Гб.

П.2.3. Подготовка системы к работе

Для начала работы с системой пользователю необходимо зарегистрироваться в системе. При регистрации каждый пользователь получает обычную роль. Для получения роли администратора нужна помощь человека, имеющего доступ к БД, чтобы вручную изменить роль пользователя.

П.2.4. Описание операций

П.2.4.1. Просмотр состояния программных сервисов

Данный процесс доступен пользователю с любой ролью. Для того чтобы начать просмотр состояния сервисов пользователю нужно пройти процесс аутентификации. В случае успеха он автоматически будет перенаправлен на страницу просмотра состояния программных сервисов.

П.2.4.2. Просмотр карточки сервиса

Данный процесс доступен пользователю с любой ролью. Для просмотра карточки сервиса пользователю необходимо нажать на самую карточку нужного

сервиса на основной странице приложения. После нажатия будет открыто модальное окно с информацией о данном сервисе.

П.2.4.3. Обновление информации о конкретном сервисе

Данный процесс доступен пользователю с любой ролью. Для обновления информации об одном сервисе необходимо нажать на значок перезагрузки внизу карточки сервиса, расположенной на основной странице.

П.2.4.4. Переход в Grafana для просмотра метрик сервиса

Данный процесс доступен пользователю с любой ролью. Для перехода в интерфейс Grafana пользователь должен открыть карточку сервиса и нажать на ссылку «Перейти в Grafana». После нажатия пользователь будет перенаправлен в интерфейс Grafana.

П.2.4.5. Обновление информации сервиса

Данный процесс доступен только пользователю с ролью администратора. Для обновления информации о сервисе пользователю необходимо перейти в карточку сервиса и нажать на значок шестеренки. После нажатия откроется модальное окно, в котором пользователь должен заполнить нужные поля и нажать кнопку внизу модального окна для обновления информации в БД.

П.2.4.6. Просмотр истории инцидентов

Данный процесс доступен пользователю с любой ролью. Для просмотра инцидентов пользователю необходимо нажать на кнопку «Инциденты» в правом верхнем углу экрана. После нажатия откроется модальное окно, в котором пользователь может задать нужные фильтры и нажать на кнопку отправки формы на серверную часть. После нажатия на кнопку пользователь увидит отфильтрованные инциденты. Если инцидентов больше 5, то он увидит на первой странице только 5 инцидентов. Для просмотра оставшихся он может переключаться по страницам при помощи соответствующих кнопок внизу модальной формы.

П.2.4.7. Обновление информации о пользователе

Данный процесс доступен только пользователю с ролью администратора. Для редактирования пользователя администратору необходимо нажать на значок шестеренки в правом нижнем углу основной страницы приложения. После нажатия на кнопку администратор по логину пользователя может найти нужного, выбрать его и указать новые данные для аутентификации, либо задать ему новую роль.